

CellScientist: Model Revision by Diagnostic Routing for Morphological Perturbation Prediction

Mengran Li^{1,2,*} Bo Li^{3,*} Jiaying Wang² Wenbin Xing² Chengyang Zhang⁴
Jinlin Wu¹ Zhen Lei¹ Jiebo Luo¹ Stan Z. Li⁵ Zelin Zang^{1,†}

¹Centre for Artificial Intelligence and Robotics,
Hong Kong Institute of Science & Innovation, Chinese Academy of Sciences

²Sun Yat-sen University ³University of Macau

⁴Sichuan University ⁵Westlake University

*Equal contribution. †Corresponding author: Zelin Zang.

ABSTRACT

Predicting how chemical perturbations reshape cellular morphology supports the comparison of compound effects through high-dimensional phenotypic readouts. Developing such predictors requires coordinated choices about chemical representation, dose conditioning, cellular context, and response modeling. Yet execution and validation feedback rarely indicate directly which choice should change next. We introduce **CellScientist**, a workflow that treats model revision as diagnostic routing: recorded discrepancies and search history guide local changes to implicated model components. Revision tracking and bounded repair connect each change to its implementation and measured outcome, while a fixed task contract preserves evaluation consistency. We evaluate this approach on three Cell Painting benchmarks with compound- and plate-based partitions. Open-workflow experiments characterize selected predictors, including search-used validation estimates. A separate controlled study compares routing policies within a fixed 768-candidate language using an independent reporting partition. Across four task/split settings, CellScientist improves held-out global Pearson correlation over random routing by 0.0361 and 0.0373 after three and five candidate evaluations, respectively; the mean gap narrows at ten. Component audits examine history, checks, and diagnostics, and transcriptomic and temporal prediction tasks assess the workflow beyond morphology. These results support diagnostic routing for finding effective morphological perturbation predictors under limited evaluation budgets. Code is available at <https://github.com/limengran98/CellScientist>.

1 INTRODUCTION

Predicting how chemical perturbations reshape cellular morphology supports the comparison of compound effects and the characterization of phenotypic responses (Seal et al., 2025). Cell Painting assays capture these responses through multivariate measurements of size, shape, texture, and fluorescence intensity (Stirling et al., 2021; Bray et al., 2017). Predictive models seek to estimate these responses for new compounds or experimental contexts. Their usefulness therefore depends on learning chemical effects across correlated responses while accounting for experimental variation (Chandrasekaran et al., 2023; Arevalo et al., 2024).

Effective predictors require coordinated choices of chemical and context representations, dose conditioning, feature interactions, objectives, and training procedures. A poor validation score does not identify which choice caused the failure. Increasing capacity may leave weak conditioning unresolved; altering a response representation can change which decoder is appropriate. Execution failures carry yet another signal about whether an implementation satisfies the task. Each evaluated candidate also consumes computational effort, making the next revision a decision about both model design and search efficiency. Model development therefore requires attributing feedback to a specific next revision.

AutoML systems search predefined configurations (Wang et al., 2021; Gijbbers et al., 2024), while LLM-based agents generate and revise executable models (Jiang et al., 2025; Lu et al., 2024; Tang et al., 2025). These capabilities motivate a further question: how should execution diagnostics, validation discrepancies, and previous attempts determine which component to revise next? Answering it requires connecting observed feedback to actionable design choices while preserving the inputs, targets, partitions, and evaluator that define the task.

We propose **CellScientist**, a workflow for model revision by diagnostic routing. It represents coupled design choices, routes feedback to implicated components, and records successive implementations and outcomes (Figure 1). A fixed task contract preserves evaluation semantics throughout the search, and each retained model is accompanied by an inspectable revision history.

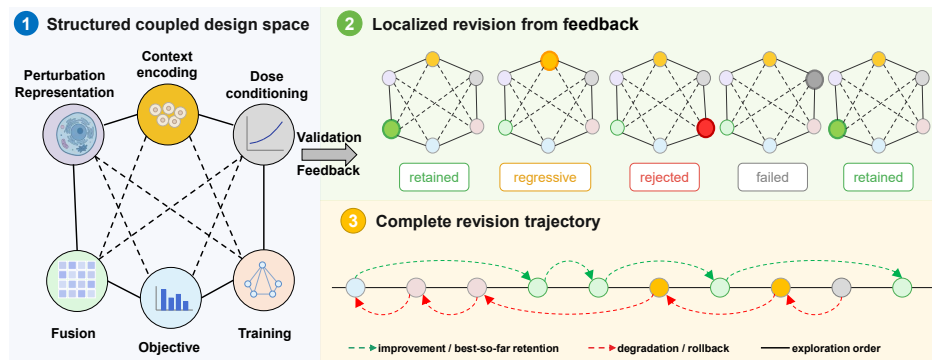


Figure 1: **Conceptual overview of CellScientist.** Revision operates over coupled choices of perturbation representation, context encoding, dose conditioning, fusion, objectives, and training strategy. Recorded feedback guides local updates, while the trajectory preserves improvements, regressions, rollbacks, rejected candidates, execution failures, and validation-retained states.

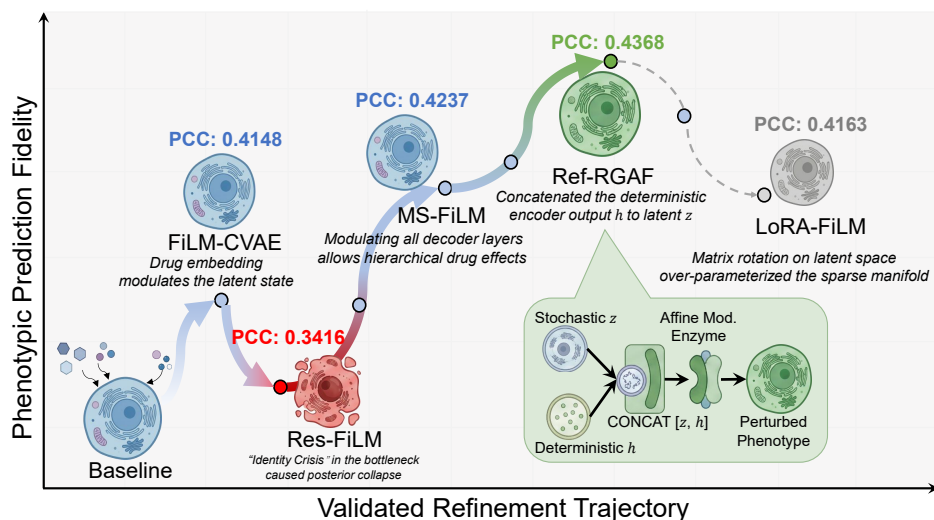


Figure 2: **Feedback-guided model-revision trajectory on BBBC047.** Recorded validation feedback guides successive revisions from modulation-based candidates to Ref-RGAF, which adds a deterministic control-reference encoding and attains the highest validation PCC in this trajectory. The annotations summarize working diagnostics recorded during the search. The retained record includes intermediate regressions and the later, lower-performing LoRA-FiLM candidate.

CellScientist implements this process through three complementary functions. History-Aware Revision Tracking (HRT) maintains design states and evidence from previous attempts. Limited-Change Application (LCA) checks candidate eligibility and applies bounded repair. Performance-Discrepancy Refinement (PDR) links observed discrepancies to revision targets. Together, they connect the ra-

tionale for an edit to its implementation and subsequent validation, while preserving unsuccessful branches for inspection.

Figure 2 illustrates this process on BBBC047. Modulation-based candidates show both improvements and regressions, prompting a working diagnosis that the supplied control-reference information is underused. A localized revision introduces a deterministic reference encoding alongside the stochastic latent state, yielding the validation-selected Ref-RGAF candidate. The recorded later regression makes clear why the selected model differs from the last attempted candidate. Appendix B.2 gives the detailed trace.

We evaluate CellScientist on three Cell Painting benchmarks, BBBC036, BBBC047, and CPG0016, using compound- and plate-based partitions. Open-workflow experiments assess selected predictors; controlled experiments compare routing policies under a shared candidate language and evaluation budget. LINCS transcriptomic prediction and Schiebinger temporal forecasting examine additional response settings.

Our contributions are threefold. First, we formulate model development for morphological perturbation prediction as feedback-guided revision of coupled design choices. Second, CellScientist connects diagnostic routing to recorded history, executable edits, and validation outcomes. Third, benchmark comparisons, controlled experiments, and component audits examine predictor quality, revision efficiency, and operational behavior.

2 RELATED WORK

Morphological and molecular perturbation prediction. Cell Painting represents chemical responses through correlated morphological measurements (Stirling et al., 2021; Bray et al., 2017; Chandrasekaran et al., 2023; Seal et al., 2025). Learning from these profiles requires attention to compound representation and experimental variation (Arevalo et al., 2024). MVCBench evaluates drug and gene representations across transcriptomic and morphological phenotypes (Li et al., 2026a); CHMR and MORSE integrate molecular structure with cellular and genomic modalities (Li et al., 2026b; Zhang et al., 2026). DeepCE, MultiDCP, and TranSiGen address transcriptomic responses (Pham et al., 2021; Wu et al., 2022; Tong et al., 2024), where batch and context variation also affect prediction (Zhou et al., 2025; Zhang et al., 2025). These complementary readouts motivate task-specific predictors; our central question is how feedback guides their revision.

Automated model development. FLAML and random-search benchmarks automate model and hyperparameter selection in predefined spaces (Wang et al., 2021; Gijssbers et al., 2024). LLM-based agents extend the proposal space through code generation and iterative revision (Jiang et al., 2025; Lu et al., 2024; Turcan et al., 2025). CellForge studies agent-designed perturbation models (Tang et al., 2025), while HarmonyCell combines hierarchical, validation-guided search with pruning, repair, and solution reuse (Huang et al., 2026). CellScientist emphasizes diagnosis- and history-conditioned revision routing. Our morphology benchmarks compare task-specific models; a shared candidate language and matched evaluation budgets support a separate comparison of routing policies.

Biomedical agents and cellular modeling. Biomedical agents support spatial biology (Wang et al., 2025a; Yu et al., 2025), omics analysis (Qiu et al., 2025), and protocol automation (Wang et al., 2025b). Their performance depends on workflow structure and reproducible evaluation (Miller et al., 2025). VCWorld integrates structured biological knowledge into virtual-cell simulation (Wei et al., 2026). We focus on developing predictors for chemical perturbations measured by morphology, with explicit design states, protected task semantics, and inspectable revision histories. LINCS and Schiebinger provide additional settings for examining this workflow under transcriptomic and temporal prediction protocols.

3 METHODOLOGY

CellScientist is a protocol-constrained controller for iterative model search. In the primary morphology task, inputs comprise a chemical descriptor, dose, and control-reference profile; targets are CellProfiler features of the perturbed sample. The controller links model-design states $\mathcal{H}$ to executable

candidates $\mathcal{I}$ through “Design Specification $\rightarrow$ Executable Candidate $\rightarrow$ Validation Feedback $\rightarrow$ Local Revision”.

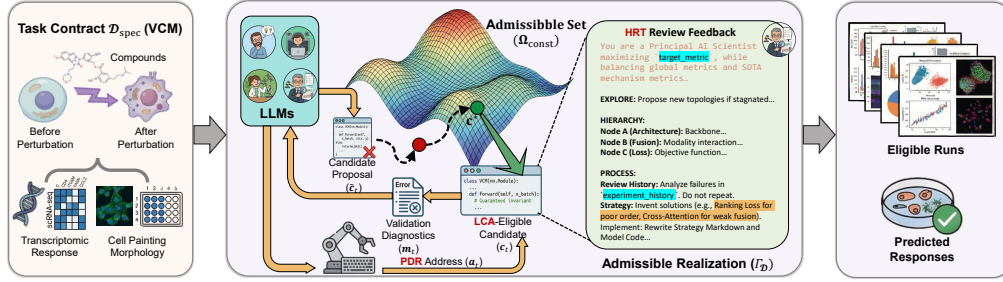


Figure 3: **Overview of the CellScientist framework.** The task contract $\mathcal{D}_{\text{spec}}$ fixes inputs, perturbation descriptors, response targets, data partitions, and evaluation semantics. HRT supplies the design state $\mathbf{h}_t$, task-state graph $\mathcal{G}_t$, and run history $\mathcal{B}_t$ for proposing $\tilde{\mathbf{c}}_t$. LCA applies configured checks and bounded repair to return an eligible candidate $\mathbf{c}_t$ or rejection $\perp$. Validation yields diagnostics $\mathbf{m}_t$, which PDR maps to a component address $\mathbf{a}_t$ for localized revision. Validation selection retains the best eligible candidate; the complete trajectory is recorded.

3.1 TASK CONTRACT AND ITERATIVE SEARCH

Let $\mathbf{h}_t \in \mathcal{H}$ denote the model-design specification at iteration t and let $\mathbf{c}_t \in \mathcal{I}$ denote an executable candidate. A task is specified by $\mathcal{D}_{\text{spec}} = (\mathcal{X}, \mathcal{P}, \mathcal{Y}, \mathcal{S}, \mathcal{E})$, where $\mathcal{X}$ contains observed cellular context, $\mathcal{P}$ contains perturbation descriptors, $\mathcal{Y}$ contains response targets, $\mathcal{S}$ defines the data-partition protocol, and $\mathcal{E}$ defines the evaluation functional. The task contract remains fixed throughout a search trajectory.

The controller implements the following operational sequence:

$$\begin{aligned} \tilde{\mathbf{c}}_t &= \Phi(\mathbf{h}_t, \mathcal{G}_t, \mathcal{B}_t), \\ \mathbf{c}_t &= \Gamma_{\mathcal{D}}(\tilde{\mathbf{c}}_t; \mathbf{h}_t, \mathcal{G}_t, \mathcal{B}_t, R) \in \Omega_{\text{const}} \cup \{\perp\}, \\ \mathbf{m}_t &= \mathcal{F}(\mathbf{c}_t; \mathcal{D}_{\text{val}}, \mathcal{E}), \\ \mathbf{h}_{t+1} &= \mathbf{h}_t \oplus \mathcal{T}_{\text{edit}}(\mathcal{A}(\mathbf{m}_t, \mathcal{G}_t, \mathcal{B}_t) \mid \mathbf{m}_t). \end{aligned} \quad (1)$$

Here Φ is the LLM-conditioned proposal operator, $\Gamma_{\mathcal{D}}$ is the configured check/repair/rejection procedure, $\mathbf{m}_t$ records execution and validation diagnostics, and $\mathcal{A}$ selects the intended revision target. The controller state is $\mathcal{Z}_t = (\mathbf{h}_t, \mathcal{G}_t, \mathcal{B}_t)$, where $\mathcal{G}_t$ is the structured task-state record and $\mathcal{B}_t$ is the run history containing attempted implementations and outcomes. Equation 1 summarizes the implemented sequential search procedure.

3.2 HRT: HISTORY-AWARE REVISION TRACKING

History-Aware Revision Tracking (HRT) represents the editable model-design state as a directed task-state graph $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$. Nodes denote components such as perturbation representation, cellular-context encoding, fusion, architecture, objective design, and training strategy; edges record configured dependencies or ordering relations. The graph provides structured context for candidate generation and is updated with task status and evidence from prior attempts.

Given a diagnostic, the controller records one or more intended revision targets in $\mathcal{V}_t$ and supplies their component descriptions to the next proposal step, making the edit scope and history inspectable. HRT also carries configured task rules, including required stages, interface expectations, and protected evaluation fields, for downstream checks.

3.3 LCA: LIMITED-CHANGE APPLICATION

Limited-Change Application (LCA) is the candidate gate between generation and validation execution. The proposal $\tilde{\mathbf{c}}_t = \Phi(\mathbf{h}_t, \mathcal{G}_t, \mathcal{B}_t)$ is applied to configured mutable notebook regions and checked

against a configured rule set $\mathcal{C}_I$. The implementation checks required interfaces and outputs, tensor-shape and numerical failures observed at runtime, and metric-field completeness. Within a fixed repair budget, failed candidates may be locally repaired and checked again; candidates that still fail are recorded as $\perp$ and excluded from validation selection.

We write the eligible set as $\Omega_{\text{const}}(\mathcal{D}_{\text{spec}}) = \{\mathbf{c} \in \mathcal{I} : \chi_I(\mathbf{c}, \mathcal{D}_{\text{spec}}) = 1\}$, where χ_I summarizes the configured checks. Operationally,

$$\mathbf{c}_t = \Gamma_{\mathcal{D}}(\tilde{\mathbf{c}}_t; \mathbf{h}_t, \mathcal{G}_t, \mathcal{B}_t, R) \in \Omega_{\text{const}} \cup \{\perp\}. \quad (2)$$

Thus, $\Gamma_{\mathcal{D}}$ covers code application, checking, bounded repair, and rejection. Passing the implemented checks establishes operational eligibility. Candidate context and outcomes are appended to $\mathcal{B}_{t+1}$ for subsequent proposals.

3.4 PDR: PERFORMANCE-DISCREPANCY REFINEMENT

Performance-Discrepancy Refinement (PDR) converts execution outcomes into diagnostic-guided model-design revisions. After executing an eligible implementation $\mathbf{c}_t$ under the validation protocol, CellScientist records feedback as

$$\begin{aligned} \mathbf{m}_t &= \mathcal{F}(\mathbf{c}_t; \mathcal{D}_{\text{val}}, \mathcal{E}) \\ &= [\psi_1(\mathbf{c}_t), \dots, \psi_{K_t}(\mathbf{c}_t)], \\ \mathbf{h}_{t+1} &= \mathbf{h}_t \oplus \mathcal{T}_{\text{edit}}(a_t \mid \mathbf{m}_t). \end{aligned} \quad (3)$$

Each diagnostic ψ_k extracts a signal from execution, validation, or rule checking. PDR maps the diagnostic to an address $a_t = \mathcal{A}(\mathbf{m}_t, \mathcal{G}_t, \mathcal{B}_t)$ identifying one or more model components; $\mathcal{T}_{\text{edit}}$ constructs the next revision request. This prompt-conditioned search heuristic uses the recorded diagnostic, task-state context, run history, and LLM-produced focus. Subsequent validation determines candidate retention under the configured selection rule.

Controlled routing operationalizes this mapping with ordered diagnostic rules. Missing chemical/dose inputs prioritize conditioning; response PCC exceeding global PCC by 0.05 prioritizes response parameterization. History filters unsuccessful addresses, and fixed preferences instantiate at most two unseen one-field edits before LLM selection (Appendix C.4).

3.5 TRAJECTORY SELECTION AND WORKFLOW

CellScientist stores each attempt in the run history. Let $\eta_t = \mathbb{I}\{\mathbf{c}_t \in \Omega_{\text{const}}(\mathcal{D}_{\text{spec}})\}$ indicate whether the proposed implementation passed the configured eligibility checks, and let $\mathcal{B}_{t+1} = \mathcal{B}_t \cup \{(\mathbf{h}_t, \tilde{\mathbf{c}}_t, \mathbf{c}_t, \mathbf{m}_t, a_t, \eta_t)\}$. The validation-selected candidate and its design state are given by

$$(\mathbf{h}_{\text{best}}, \mathbf{c}_{\text{best}}) = \text{Select}_{\text{val}}(\{(\mathbf{h}_i, \mathbf{c}_i, \mathbf{m}_i) : \eta_i = 1\}_{i=0}^{T-1}, \mathcal{E}), \quad (4)$$

which returns the best eligible model-design state and executable candidate under the configured validation rule.

4 EXPERIMENTS

We evaluate CellScientist at two levels. The open workflow develops task-specific predictors, denoted ‘‘CS-model’’, under the morphology benchmark protocols. Its selected-model results include validation estimates used during search. A separate controlled audit fixes the candidate language, predictor, fitting procedure, data roles, and evaluation budget, measuring controller behavior on a reporting fold excluded from search. Both preserve an experiment-specific task contract and retain unsuccessful attempts.

The experiments address two questions: **RQ1: Selected-model benchmark performance.** How do the task-specific predictors selected by CellScientist compare with the listed baselines under common data and evaluation protocols? **RQ2: Candidate-evaluation efficiency.** Under matched candidate language, fitting, data roles, and evaluation budget, does diagnostic- and history-conditioned routing improve held-out PCC at small budgets, and what operational roles do HRT, LCA, and PDR play?

Table 1: **Core morphology-response benchmarks.** Five-fold mean $\pm$ sample standard deviation compares task-specific **CS-model** predictors with fixed, search-based, and agent-designed methods on BBBC036, BBBC047, and CPG0016. The two partitions group exact SMILES or experimental plates; metrics are global MSE, PCC, and R^2 . Preprocessing, model families, search, and compute may differ. The BBBC047 SMILES CS-model scores are search-used validation estimates shared with the case study (Appendix C.1); controller comparisons use separate held-out reporting (Table 3). Best means are **bolded**.

Method	Venue	SMILES-based Split			Plate-based Split		
		MSE↓	PCC↑	R ² ↑	MSE↓	PCC↑	R ² ↑
BBBC036							
# Existing reference methods							
TabR	ICLR'24	3.5518±0.24	0.2747±0.03	0.0119±0.02	2.8759±0.13	0.5065±0.03	0.1855±0.02
RealMLP	NeurIPS'24	3.5305±0.16	0.3301±0.01	0.0049±0.02	2.5740±0.16	0.5856±0.02	0.2530±0.03
RF-TD	NeurIPS'24	3.5951±0.19	0.2447±0.01	0.0035±0.01	3.5978±0.14	0.2466±0.02	0.0152±0.01
□ Search-based baselines							
FLAML	MLSys'21	3.4217±0.18	0.3486±0.02	0.0379±0.02	2.4918±0.14	0.5987±0.02	0.2784±0.03
Random Search	JMLR'24	3.4763±0.21	0.3392±0.02	0.0246±0.02	2.5335±0.15	0.5904±0.02	0.2649±0.03
△ Agent-designed methods							
CellForge	arXiv'25	3.4568±0.19	0.3415±0.03	0.0464±0.02	2.5187±0.14	0.6013±0.02	0.2878±0.03
CS-model	Ours	3.3115±0.20	0.3688±0.03	0.0717±0.02	2.3662±0.15	0.6212±0.02	0.3165±0.03
BBBC047							
# Existing reference methods							
TabR	ICLR'24	2.9382±0.04	0.3552±0.01	0.0059±0.01	2.7930±0.14	0.4094±0.01	0.0568±0.03
RealMLP	NeurIPS'24	2.8441±0.03	0.3985±0.01	0.0285±0.01	2.6475±0.10	0.4597±0.01	0.1332±0.02
RF-TD	NeurIPS'24	2.8850±0.05	0.3746±0.01	0.0281±0.01	2.8174±0.16	0.4005±0.01	0.0615±0.01
□ Search-based baselines							
FLAML	MLSys'21	2.8126±0.06	0.4127±0.01	0.0435±0.01	2.6158±0.11	0.4698±0.01	0.1407±0.02
Random Search	JMLR'24	2.8314±0.07	0.4059±0.01	0.0358±0.01	2.6342±0.12	0.4635±0.01	0.1362±0.02
△ Agent-designed methods							
CellForge	arXiv'25	2.7869±0.09	0.4154±0.01	0.0476±0.01	2.5908±0.10	0.4772±0.02	0.1471±0.02
CS-model	Ours	2.7612±0.11	0.4368±0.01	0.0687±0.01	2.5704±0.11	0.4844±0.02	0.1538±0.03
CPG0016							
# Existing reference methods							
TabR	ICLR'24	2.2622±0.12	0.0770±0.01	-0.5516±0.04	2.0667±0.04	0.1953±0.01	-0.4128±0.02
RealMLP	NeurIPS'24	1.4174±0.08	0.2616±0.02	0.0297±0.01	1.2105±0.01	0.4418±0.01	0.1717±0.01
RF-TD	NeurIPS'24	1.4673±0.09	0.1354±0.01	-0.0035±0.01	1.3400±0.01	0.3213±0.01	0.0849±0.01
□ Search-based baselines							
FLAML	MLSys'21	1.2865±0.05	0.3748±0.01	0.1183±0.01	1.1206±0.03	0.5037±0.02	0.2331±0.02
Random Search	JMLR'24	1.3428±0.06	0.3189±0.02	0.0747±0.02	1.1674±0.03	0.4668±0.02	0.1994±0.02
△ Agent-designed methods							
CellForge	arXiv'25	1.2849±0.03	0.3716±0.01	0.1272±0.01	1.0458±0.02	0.5187±0.03	0.2376±0.03
CS-model	Ours	1.1113±0.01	0.5093±0.01	0.2383±0.01	0.9937±0.02	0.5761±0.04	0.3117±0.04

4.1 EXPERIMENTAL SETUP

Cell Painting morphology is the primary task; LINCS transcriptomic prediction and Schiebinger temporal forecasting provide extensions. Dataset definitions, baselines, metrics, and implementation details appear in Appendices C.1–C.4.

Cell Painting morphology prediction. We evaluate three public high-content screening datasets in CellProfiler feature space (Stirling et al., 2021): BBBC036 and BBBC047 (Bray et al., 2017), and CPG0016 (Chandrasekaran et al., 2023; Arevalo et al., 2024). All three morphology benchmarks are evaluated under paired SMILES-based and plate-based splits. Baselines for the main morphology table include TabR (Gorishniy et al., 2024), RealMLP and RF-TD (Holzmüller et al., 2024), FLAML (Wang et al., 2021), Random Search (Gijsbers et al., 2024), and CellForge (Tang et al., 2025). The main text reports global MSE, PCC, and R^2 ; top- K response-feature morphology metrics are provided in Appendix F.2.

Controlled framework audit. The audit uses BBBC036/047 under both plate- and SMILES-based settings. All policies share the same initial state, 768-candidate executable language, predictor, fitting procedure, data roles, evaluator, LLM settings, paired seeds, candidate-retention rule, and candidate-evaluation budget. Folds 1–3 fit candidates, disjoint fold-4 group subsets provide search feedback and post-search candidate selection, and fold 5 is used only for held-out reporting. Each paired seed initializes the same randomized low-rank fitting procedure in every policy, so the reported dispersion includes fitting as well as controller variation without adding noise to evaluated scores. After

search, each budget prefix retains the candidate maximizing mean feedback/selection PCC; routing uses feedback alone. We evaluate five paired seeds at $B \in \{1, 3, 5, 10\}$, with global PCC registered as the selection and held-out reporting endpoint. Fixed h_0 , Flat retry, Random routing, Full CellScientist, and an AIDE-style shared-language policy differ in controller policy rather than candidate family or evaluation count.

Transcriptomic perturbation prediction. For LINCS2020 L1000 perturbation prediction (Subramanian et al., 2017; Xie et al., 2022), we evaluate two settings: Seven Cell Lines and Full Data. Baselines include DLEPS (Zhu et al., 2021), DeepCE (Pham et al., 2021), CIGER (Pham et al., 2022), MultiDCP (Wu et al., 2022), and TranSiGen (Tong et al., 2024). Metrics include RMSE, PCC, and Positive/Negative P@100.

Temporal forecasting. Schiebinger reprogramming data support a separate comparison with scN-ODE and a MIOFlow adaptation. Methods share the development window, refit cutoff, and untouched final evaluation units (Appendix F.3).

Table 2: **LINCS2020 transcriptomic perturbation prediction.** CS-model achieves the best mean results among the listed methods in both the Seven Cell Lines and Full Data settings. Each uses an independent task-specific search. Best means are **bolded**.

Dataset	Model	RMSE ↓	PCC ↑	Positive P@100 ↑	Negative P@100 ↑
Seven Cell	DLEPS	1.4954 $\pm$ 0.04	0.4268 $\pm$ 0.02	0.2460 $\pm$ 0.01	0.3087 $\pm$ 0.01
	DeepCE	1.7728 $\pm$ 0.03	0.4121 $\pm$ 0.01	0.2282 $\pm$ 0.02	0.2713 $\pm$ 0.02
	CIGER	4.4646 $\pm$ 1.06	0.4165 $\pm$ 0.01	0.2337 $\pm$ 0.02	0.2849 $\pm$ 0.02
	MultiDCP	1.7697 $\pm$ 0.03	0.4461 $\pm$ 0.01	0.2378 $\pm$ 0.02	0.2872 $\pm$ 0.01
	TranSiGen	0.6903 $\pm$ 0.02	0.5332 $\pm$ 0.02	0.3785 $\pm$ 0.01	0.3886 $\pm$ 0.02
	CS-model	0.5088$\pm$0.02	0.6183$\pm$0.02	0.4450$\pm$0.02	0.4481$\pm$0.02
Full Data	TranSiGen	0.5186 $\pm$ 0.01	0.6076 $\pm$ 0.01	0.4353 $\pm$ 0.01	0.4403 $\pm$ 0.01
	CS-model	0.4967$\pm$0.01	0.6368$\pm$0.01	0.4583$\pm$0.01	0.4652$\pm$0.01

4.2 SELECTED-MODEL BENCHMARK PERFORMANCE (RQ1)

Table 1 characterizes morphology predictors, while LINCS and Schiebinger test the workflow in additional response spaces under their respective protocols.

Core morphology-response benchmarks. Table 1 reports the best mean MSE, PCC, and R^2 for CS-model among the listed methods under both partitions. On BBBC047-SMILES, Ref-RGAF attains five-fold validation PCC 0.4368 ± 0.01 ; these folds also guide early stopping, candidate selection, and subsequent revisions. The main table and Figure 2 use the same selected-model evaluation, documenting benchmark performance and its search history (Appendix C.1). Figure 4 summarizes paired-fold R^2 descriptively; Appendix F.2 reports response-feature metrics. Independent controller evaluation is provided by the controlled audit below.

Applicability across response spaces. Table 2 reports separately selected LINCS predictors; Appendix F.3 reports Schiebinger temporal forecasting. The different response spaces and evaluation units test whether the workflow can develop task-specific models beyond morphology. These results complement the primary task and retain their own protocol-specific interpretation.

4.3 MATCHED-BUDGET FRAMEWORK AUDIT (RQ2)

The controlled audit fixes candidate freedom to measure the contribution of the CellScientist controller. Policies share h_0 , the 768-candidate executable language, low-rank predictor, fitting procedure, fold roles, diagnostic record, and candidate-evaluation budget. This predictor differs from the task-specific CS-model architectures in Table 1. Global PCC, computed after flattening prediction and observation matrices, is the sole metric for validation-based candidate retention and held-out frontier reporting. The full feedback-metric record remains available for diagnostic component attribution.

Matched policies. Fixed h_0 performs no revision. Flat retry selects among up to 12 unseen one-step neighbors without a typed address. Random routing samples uniformly among addresses with unseen

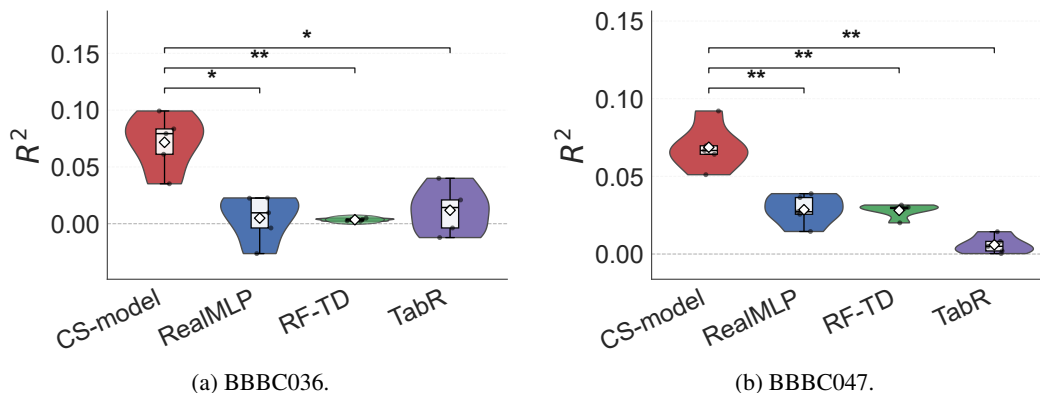


Figure 4: **Fold-level comparison on BBBC036 and BBBC047.** Five-fold R^2 distributions compare **CS-model** with reference predictors. Violins show distributions; boxes, points, and diamonds denote quartiles, folds, and means. Stars give descriptive, uncorrected paired t -tests on the benchmark folds: * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.

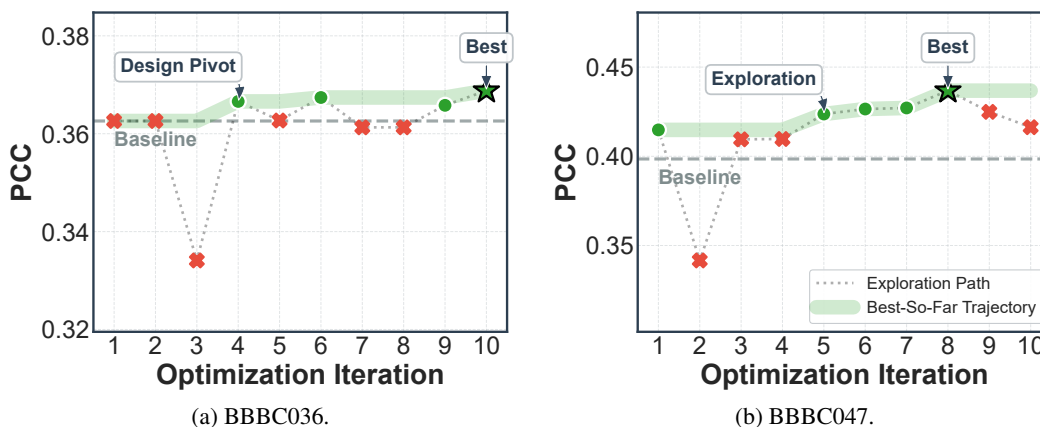


Figure 5: **High-freedom workflow trajectories.** Raw validation utilities and retained best-so-far trajectories over ten refinement steps under the SMILES-based split. Exploratory regressions remain visible alongside the validation-selected path.

neighbors, then uses the same deterministic edit instantiator as CellScientist. AIDE-style search allocates at most five independent drafts before greedy improvement, within total budget B . CellScientist applies ordered diagnostic rules, filters addresses using history, and instantiates one unseen neighbor for each of at most two addresses. Gemini 3 Pro selects one allowed address–candidate pair; invalid or failed responses fall back to the first pair. Appendix C.4 specifies every rule and preference. The comparison measures the combined routing policy, including its diagnostic and history rules.

Controller attribution. Table 3 supports higher candidate-evaluation efficiency at small budgets. Across four task/split settings, CellScientist exceeds Random routing by 0.0361 PCC at $B = 3$ (95% CI [0.0275, 0.0448]) and 0.0373 at $B = 5$ ([0.0292, 0.0454]). Budgets include h_0 , so these allow two and four revisions. At $B = 10$, the mean gap narrows to 0.0043 ([−0.0018, 0.0104]), leaving the aggregate difference unresolved. CellScientist retains a 0.0181 advantage over AIDE-style search at $B = 10$ ([0.0052, 0.0309]). Its normalized frontier-AUC advantages over Fixed h_0 , Flat retry, Random routing, and AIDE-style search are 0.0877, 0.0608, 0.0237, and 0.0215, respectively, with all four reported confidence intervals above zero.

Appendix D reports backbone-specific results. Qwen2.5-0.5B-Instruct reproduces positive CellScientist-minus-Random effects of 0.0364 and 0.0341 at $B = 3$ and $B = 5$, with a frontier-AUC difference of 0.0217 (95% CI [0.0158, 0.0275]). These results support routing efficiency with both tested backbones within the fixed candidate language.

Table 3: **Five-seed matched-budget controller comparison on held-out global PCC.** The upper panel reports mean $\pm$ sample standard deviation over five paired controller and fitting seeds. The lower panel reports CellScientist-minus-comparator effects with paired-setting 95% t -confidence intervals across the four task/split settings, computed from unrounded run-level values; confidence intervals are shown in smaller type. All policies use the same initial candidate, executable candidate language, data roles, fitting procedure, and number of evaluated candidates. The uninformative common- h_0 row at $B = 1$ is omitted. Best means at each task, split, and budget are **bolded**.

Dataset	Split	Budget	Fixed h_0	Flat retry	Random routing	AIDE-style	CellScientist
BBBC036	Plate	3	0.1664 $\pm$ 0.0005	0.1828 $\pm$ 0.0005	0.2250 $\pm$ 0.0493	0.2244 $\pm$ 0.0276	0.2650 $\pm$ 0.0005
		5	0.1664 $\pm$ 0.0005	0.2073 $\pm$ 0.0056	0.2277 $\pm$ 0.0502	0.2477 $\pm$ 0.0294	0.2674 $\pm$ 0.0003
		10	0.1664 $\pm$ 0.0005	0.2175 $\pm$ 0.0061	0.2626 $\pm$ 0.0077	0.2562 $\pm$ 0.0205	0.2693 $\pm$ 0.0012
BBBC036	SMILES	3	0.1547 $\pm$ 0.0005	0.1583 $\pm$ 0.0046	0.2052 $\pm$ 0.0413	0.2058 $\pm$ 0.0230	0.2336 $\pm$ 0.0016
		5	0.1547 $\pm$ 0.0005	0.1882 $\pm$ 0.0129	0.2049 $\pm$ 0.0417	0.2231 $\pm$ 0.0169	0.2346 $\pm$ 0.0009
		10	0.1547 $\pm$ 0.0005	0.1948 $\pm$ 0.0005	0.2347 $\pm$ 0.0016	0.2235 $\pm$ 0.0164	0.2346 $\pm$ 0.0009
BBBC047	Plate	3	0.2442 $\pm$ 0.0008	0.2500 $\pm$ 0.0055	0.3032 $\pm$ 0.0500	0.3013 $\pm$ 0.0381	0.3396 $\pm$ 0.0004
		5	0.2442 $\pm$ 0.0008	0.2630 $\pm$ 0.0130	0.3066 $\pm$ 0.0514	0.3153 $\pm$ 0.0399	0.3457 $\pm$ 0.0004
		10	0.2442 $\pm$ 0.0008	0.2777 $\pm$ 0.0005	0.3458 $\pm$ 0.0027	0.3191 $\pm$ 0.0372	0.3481 $\pm$ 0.0009
BBBC047	SMILES	3	0.2504 $\pm$ 0.0004	0.2651 $\pm$ 0.0002	0.3131 $\pm$ 0.0536	0.3123 $\pm$ 0.0346	0.3529 $\pm$ 0.0010
		5	0.2504 $\pm$ 0.0004	0.2916 $\pm$ 0.0003	0.3207 $\pm$ 0.0582	0.3371 $\pm$ 0.0417	0.3614 $\pm$ 0.0003
		10	0.2504 $\pm$ 0.0004	0.2920 $\pm$ 0.0003	0.3668 $\pm$ 0.0073	0.3560 $\pm$ 0.0359	0.3751 $\pm$ 0.0017
CellScientist advantage: mean effect with paired-setting 95% CI							
		Budget		vs. Fixed h_0	vs. Flat	vs. Random	vs. AIDE-style
		B = 3		+0.0939 [+0.0773,+0.1104]	+0.0837 [+0.0734,+0.0940]	+0.0361 [+0.0275,+0.0448]	+0.0368 [+0.0270,+0.0466]
		B = 5		+0.0984 [+0.0774,+0.1193]	+0.0648 [+0.0404,+0.0892]	+0.0373 [+0.0292,+0.0454]	+0.0215 [+0.0088,+0.0342]
		B = 10		+0.1029 [+0.0737,+0.1320]	+0.0612 [+0.0306,+0.0919]	+0.0043 [-0.0018,+0.0104]	+0.0181 [+0.0052,+0.0309]

HRT mechanism. Table 4a removes only persistent proposal history while fixing the trained-candidate budget. HRT eliminates repeated proposals and more than doubles the unique-proposal ratio. Under HRT w/o History, repeated IDs are recorded and replaced by legal unseen candidates before evaluation, preserving the matched budget. The ablation measures the additional unique-proposal coverage provided by persistent history.

LCA and PDR mechanisms. Table 4, panels b and c, summarizes the registered LCA and held-out PDR audits. LCA correctly classifies all 60 registered cases and contains all 40 violations. Structured PDR localizes and repairs all 15 held-out faults, with no unrelated edits and one repair round on average. Appendix E provides outcome-level and component-address breakdowns. These audits complement the predictive frontier in Table 3 with direct measurements of operational behavior.

Table 4: **Compact HRT, LCA, and PDR mechanism audits.** (a) HRT compares 20 matched Gemini-controlled runs with 200 candidate evaluations; HRT and w/o History use 170 and 180 proposal steps. HRT w/o History removes only the persistent seen/failed record while retaining the current state, diagnostics, candidate language, and controller. (b) LCA summarizes 40 violations and 20 legal controls; it makes 60/60 correct decisions, contains 40/40 violations, and avoids 30 invalid evaluations. (c) PDR compares Flat retry, Random routing, and Structured PDR over 15 held-out faults. Full LCA and PDR breakdowns appear in Appendix E; best values are **bolded**.

(a) HRT history.			(b) LCA outcomes.			(c) PDR held-out results.				
Measure	HRT	w/o Hist.	Outcome	Cases	Result	Policy	Top-1	Repair	Edits	Rounds
Unique proposals $\uparrow$	170	82	Repair	15	15/15 repaired	Flat	13.3%	33.3%	3.20	4.80
Repeated proposals $\downarrow$	0	98	Pre-run block	15	15/15 blocked	Random	6.7%	60.0%	3.27	4.27
Repeat rate $\downarrow$	0.0%	54.4%	Post-run block	10	10/10 blocked	Structured	100%	100%	0.00	1.00
Unique ratio $\uparrow$	100.0%	45.6%	Legal controls	20	20/20 accepted					

5 CONCLUSION

CellScientist makes feedback-guided model revision explicit for morphological perturbation prediction, connecting chemical and context modeling choices to auditable search histories. Open-workflow benchmarks characterize selected predictors; controlled experiments show more efficient candidate evaluation at small budgets, with a diminishing mean advantage over random routing at ten evaluations. LINC and Schiebinger extend the workflow to additional response settings. These results support finding useful revisions with fewer candidate evaluations in the studied morphology tasks. The resulting trajectories make each selected model’s development traceable to concrete edits, execution outcomes, and validation decisions.

REPRODUCIBILITY STATEMENT

The operational algorithm and notation appear in Appendix A; Appendix C.4 specifies the diagnostic rules, history filters, deterministic edits, and LLM decisions. Dataset definitions, baseline protocols, evaluation metrics, implementation settings, and controller configurations are detailed in Appendices C.1–C.4. The case studies retain failed and regressive candidates alongside validation-selected outcomes, while the registered LCA and held-out PDR audits are documented in Appendix E.

REFERENCES

- John Arevalo, Ellen Su, Jessica D. Ewald, Robert van Dijk, Anne E. Carpenter, and Shantanu Singh. Evaluating batch correction methods for image-based cell profiling. *Nature Communications*, 15: 6516, 2024. doi: 10.1038/s41467-024-50613-5.
- Mark-Anthony Bray, Sigrun M. Gustafsdottir, Mohammad H. Rohban, Shantanu Singh, Vebjorn Ljosa, Katherine L. Sokolnicki, Joshua A. Bittker, et al. A dataset of images and morphological profiles of 30,000 small-molecule treatments using the Cell Painting assay. *GigaScience*, 6(12): 1–5, 2017. doi: 10.1093/gigascience/giw014.
- Srinivas Niranj Chandrasekaran, Jeanelle Ackerman, Eric Alix, D Michael Ando, John Arevalo, Melissa Bennion, Nicolas Boisseau, Adriana Borowa, Justin D Boyd, Laurent Brino, et al. Jump cell painting dataset: morphological impact of 136,000 chemical and genetic perturbations. *BioRxiv*, 2023. doi: 10.1101/2023.03.23.534023.
- Pieter Gijsbers, Marcos LP Bueno, Stefan Coors, Erin LeDell, Sébastien Poirier, Janek Thomas, Bernd Bischl, and Joaquin Vanschoren. Amlb: an automl benchmark. *Journal of Machine Learning Research*, 25(101):1–65, 2024. URL <https://www.jmlr.org/papers/v25/22-0493.html>.
- Yury Gorishniy, Ivan Rubachev, Nikolay Kartashev, Daniil Shlenskii, Akim Kotelnikov, and Artem Babenko. Tabr: Tabular deep learning meets nearest neighbors. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=rhgIgTSSxW>.
- David Holzmüller, Léo Grinsztajn, and Ingo Steinwart. Better by default: Strong pre-tuned mlps and boosted trees on tabular data. *Advances in Neural Information Processing Systems*, 37:26577–26658, 2024. doi: 10.52202/079017-0837.
- Wenxuan Huang, Mingyu Tsoi, Yanhao Huang, Xinjie Mao, Xue Xia, Hao Wu, Jiaqi Wei, Yuejin Yang, Lang Yu, Cheng Tan, et al. HarmonyCell: Automating single-cell perturbation modeling under semantic and distribution shifts. *arXiv preprint arXiv:2603.01396*, 2026.
- Guillaume Hugué, Daniel Sumner Magruder, Alexander Tong, Oluwadamilola Fasina, Manik Kuchroo, Guy Wolf, and Smita Krishnaswamy. Manifold interpolating optimal-transport flows for trajectory inference. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. Aide: Ai-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025. URL <https://arxiv.org/abs/2502.13138>.
- Bo Li, Qing Wang, Shihang Wang, Bob Zhang, Yuzhong Peng, Pinxian Zeng, Chengliang Liu, Mengran Li, Ziyang Tang, Xiaojun Yao, Chuxia Deng, and Qianqian Song. MVCBench: A multimodal benchmark for drug-induced virtual cell phenotypes. *bioRxiv*, 2026a. doi: 10.64898/2026.04.22.720110. URL <https://www.biorxiv.org/content/10.64898/2026.04.22.720110v1>. Preprint, version 1.
- Mengran Li, Zelin Zang, Wenbin Xing, Junzhou Chen, Ronghui Zhang, Jiebo Luo, and Stan Z. Li. Learning cell-aware hierarchical multi-modal representations for robust molecular modeling. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 623–631, 2026b. doi: 10.1609/aaai.v40i1.37027.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024. URL <https://arxiv.org/abs/2408.06292>.

- Henry E Miller, Matthew Greenig, Benjamin Tenmann, and Bo Wang. Bioml-bench: Evaluation of ai agents for end-to-end biomedical ml. *bioRxiv*, 2025. doi: 10.1101/2025.09.01.673319.
- Thai-Hoang Pham, Yue Qiu, Jucheng Zeng, Lei Xie, and Ping Zhang. A deep learning framework for high-throughput mechanism-driven phenotype compound screening and its application to covid-19 drug repurposing. *Nature Machine Intelligence*, 3:247–257, 2021. doi: 10.1038/s42256-020-00285-9.
- Thai-Hoang Pham, Yue Qiu, Jiahui Liu, Steven Zimmer, Eric O’Neill, Lei Xie, and Ping Zhang. Chemical-induced gene expression ranking and its application to pancreatic cancer drug repurposing. *Patterns*, 3(4):100441, 2022. doi: 10.1016/j.patter.2022.100441.
- Yibo Qiu, Zan Huang, Zhiyu Wang, Handi Liu, Yiling Qiao, Yifeng Hu, Shu’ang Sun, Hangke Peng, Ronald X Xu, and Mingzhai Sun. Biomars: A multi-agent robotic system for autonomous biological experiments. *arXiv preprint arXiv:2507.01485*, 2025. URL <https://arxiv.org/abs/2507.01485>.
- Geoffrey Schiebinger, Jian Shu, Marcin Tabaka, Brian Cleary, Vidya Subramanian, Aryeh Solomon, Joshua Gould, Siyan Liu, Stacie Lin, Peter Berube, Lia Lee, Jenny Chen, Justin Brumbaugh, Philippe Rigollet, Konrad Hochedlinger, Rudolf Jaenisch, Aviv Regev, and Eric S. Lander. Optimal-transport analysis of single-cell gene expression identifies developmental trajectories in reprogramming. *Cell*, 176(4):928–943.e22, 2019. doi: 10.1016/j.cell.2019.01.006.
- Srijit Seal, Maria-Anna Trapotsi, Ola Spjuth, Shantanu Singh, Jordi Carreras-Puigvert, Nigel Greene, Andreas Bender, and Anne E Carpenter. Cell painting: a decade of discovery and innovation in cellular imaging. *Nature methods*, 22(2):254–268, 2025. doi: 10.1038/s41592-024-02528-8.
- Vasileios Stathias, John Turner, Amar Koleti, Dusica Vidovic, Daniel Cooper, Mehdi Fazel-Najafabadi, Marcin Pilarczyk, Raymond Terryn, Caty Chung, Afoma Umeano, et al. Lincs data portal 2.0: next generation access point for perturbation-response signatures. *Nucleic Acids Research*, 48(D1):D431–D439, 2020. doi: 10.1093/nar/gkz1023.
- David R Stirling, Madison J Swain-Bowden, Alice M Lucas, Anne E Carpenter, Beth A Cimini, and Allen Goodman. Cellprofiler 4: improvements in speed, utility and usability. *BMC bioinformatics*, 22(1):433, 2021. doi: 10.1186/s12859-021-04344-9.
- Aravind Subramanian, Rajiv Narayan, Steven M Corsello, David D Peck, Ted E Natoli, Xiaodong Lu, Joshua Gould, John F Davis, Andrew A Tubelli, Jacob K Asiedu, et al. A next generation connectivity map: L1000 platform and the first 1,000,000 profiles. *Cell*, 171(6):1437–1452.e17, 2017. doi: 10.1016/j.cell.2017.10.049.
- Xiangru Tang, Zhuoyun Yu, Jiapeng Chen, Yan Cui, Daniel Shao, Weixu Wang, Fang Wu, Yuchen Zhuang, Wenqi Shi, Zhi Huang, et al. Cellforge: agentic design of virtual cell models. *arXiv preprint arXiv:2508.02276*, 2025. URL <https://arxiv.org/abs/2508.02276>.
- Xiaochu Tong, Ning Qu, Xiangtai Kong, Shengkun Ni, Jingyi Zhou, Kun Wang, Lehan Zhang, Yiming Wen, Jiangshan Shi, Sulin Zhang, et al. Deep representation learning of chemical-induced transcriptional profile for phenotype-based drug discovery. *Nature Communications*, 15(1):5378, 2024. doi: 10.1038/s41467-024-49620-3.
- Alistair Turcan, Kexin Huang, Lei Li, and Martin JinYE Zhang. Tusoai: Agentic optimization for scientific methods. *arXiv preprint arXiv:2509.23986*, 2025. URL <https://arxiv.org/abs/2509.23986>.
- Chi Wang, Qingyun Wu, Markus Weimer, and Erkang Zhu. Flaml: A fast and lightweight auttml library. In *Proceedings of Machine Learning and Systems*, volume 3, pp. 434–447, 2021. URL https://proceedings.mlsys.org/paper_files/paper/2021/hash/1ccc3bfa05cb37b917068778f3c4523a-Abstract.html.
- Hanchen Wang, Yichun He, Paula P Coelho, Matthew Bucci, Abbas Nazir, Bob Chen, Linh Trinh, Serena Zhang, Kexin Huang, Vineethkrishna Chandrasekar, et al. Spatialagent: An autonomous ai agent for spatial biology. *bioRxiv*, pp. 2025–04, 2025a. doi: 10.1101/2025.04.03.646459.

- Wenbo Wang, Simran Swain, Jaeyong Lee, Zuwan Lin, Bradley Canales, Almir Aljovic, Yaxuan Liu, Qiang Li, Arnau Marin-Llobet, Mai Liu, et al. Agentic lab: An agentic-physical ai system for cell and organoid experimentation and manufacturing. *bioRxiv*, 2025b. doi: 10.1101/2025.11.11.686354.
- Zhijian Wei, Runze Ma, Zichen Wang, Zhongmin Li, Shuotong Song, and Shuangjia Zheng. Vcworld: A biological world model for virtual cell simulation. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=hhq89Hs7T3>. <https://openreview.net/forum?id=hhq89Hs7T3>.
- You Wu, Qiao Liu, Yue Qiu, and Lei Xie. Deep learning prediction of chemical-induced dose-dependent and context-specific multiplex phenotype responses and its application to personalized alzheimer's disease drug repurposing. *PLOS Computational Biology*, 18(8):e1010367, 2022. doi: 10.1371/journal.pcbi.1010367.
- Zhuorui Xie, Eryk Kropiwnicki, Megan L Wojciechowicz, Kathleen M Jagodnik, Ingrid Shu, Allison Bailey, Daniel JB Clarke, Minji Jeon, John Erol Evangelista, Maxim V. Kuleshov, et al. Getting started with lincs datasets and tools. *Current protocols*, 2(7):e487, 2022. doi: 10.1002/cpz1.487.
- Xi Yu, Yang Yang, Qun Liu, Yonghua Du, Sean McSweeney, and Yuewei Lin. Gencellagent: Generalizable, training-free cellular image segmentation via large language model agents. *arXiv preprint arXiv:2510.13896*, 2025. URL <https://arxiv.org/abs/2510.13896>.
- Jiaqi Zhang, Erica Larschan, Jeremy Bigness, and Ritambhara Singh. scnode: generative model for temporal single cell transcriptomic data prediction. *Bioinformatics*, 40:ii146–ii154, 2024.
- Ronghui Zhang, Mengran Li, Wenbin Xing, Bo Li, Chengyang Zhang, Wenxuan Tu, Yongfu Li, and Ruxin Wang. MORSE: Molecular representation learning via structured semantic extraction across hierarchical and asymmetric biological modalities. *Pattern Recognition*, 174:112975, 2026. doi: 10.1016/j.patcog.2025.112975.
- Zhaojun Zhang, Divij Mathew, Tristan L. Lim, Kaishu Mason, Clara Morral Martinez, Sijia Huang, E. John Wherry, Katalin Susztak, Andy J. Minn, Zongming Ma, et al. Recovery of biological signals lost in single-cell batch integration with cellanova. *Nature Biotechnology*, 43:1861–1877, 2025. doi: 10.1038/s41587-024-02463-1.
- Yang Zhou, Qiongyu Sheng, Guohua Wang, Li Xu, and Shuilin Jin. Quantifying batch effects for individual genes in single-cell data. *Nature Computational Science*, 5:612–620, 2025. doi: 10.1038/s43588-025-00824-7.
- Jie Zhu, Jingxiang Wang, Xin Wang, Mingjing Gao, Bingbing Guo, Miaomiao Gao, Jiarui Liu, Yanqiu Yu, Liang Wang, Weikaixin Kong, et al. Prediction of drug efficacy from transcriptional profiles with deep learning. *Nature biotechnology*, 39(11):1444–1452, 2021. doi: 10.1038/s41587-021-00946-z.

A ALGORITHM AND NOTATION

Coupled model-design revision. Perturbation-response models combine interdependent choices in perturbation representation, cellular-context encoding, dose conditioning, feature interaction, objective design, and optimization. Changing one component can alter the behavior of others. Model revision therefore requires associating observed outcomes with candidate explanations and possible edit targets. An explicit record of component dependencies, accumulated evidence, and edit scope supports this process without establishing causal attribution.

Hypothesis–implementation alignment. Evaluating a model-design hypothesis requires an executable implementation that preserves the task’s data, prediction target, interface, and evaluation semantics. Execution and validation outcomes then inform which design assumptions and components merit reconsideration. We formulate this as a *model-design revision-routing problem*: mapping observed discrepancies to local design revisions while maintaining a fixed task contract.

Algorithm 1: CellScientist Workflow

Require: Fixed task specification $\mathcal{D}_{\text{spec}}$, initial model-design state $\mathbf{h}_0$, task-state graph $\mathcal{G}_0$, iteration budget T , repair budget R

- 1: Initialize run history $\mathcal{B}_0 \leftarrow \emptyset$ and best eligible state $(\mathbf{h}_{\text{best}}, \mathbf{c}_{\text{best}}) \leftarrow (\mathbf{h}_0, \perp)$
- 2: **for** $t = 0$ to $T - 1$ **do**
- 3: Summarize the current model-design state, task-state graph, and run history
- 4: Request a focus area and candidate edits $\tilde{\mathbf{c}}_t \leftarrow \Phi(\mathbf{h}_t, \mathcal{G}_t, \mathcal{B}_t)$
- 5: Apply edits and configured checks with at most R repair attempts: $\mathbf{c}_t \leftarrow \Gamma_{\mathcal{D}}(\tilde{\mathbf{c}}_t; \mathbf{h}_t, \mathcal{G}_t, \mathcal{B}_t, R)$
- 6: **if** $\mathbf{c}_t \neq \perp$ **then**
- 7: Execute $\mathbf{c}_t$ under the validation view of $\mathcal{D}_{\text{spec}}$
- 8: Record execution status, configured rule checks, and validation feedback $\mathbf{m}_t$
- 9: Set $\eta_t \leftarrow 1$
- 10: **else**
- 11: Record execution or required-output failure in $\mathbf{m}_t$ and set $\eta_t \leftarrow 0$
- 12: **end if**
- 13: Route the diagnostic to an edit address $a_t \leftarrow \mathcal{A}(\mathbf{m}_t, \mathcal{G}_t, \mathcal{B}_t)$
- 14: Form the next design record $\mathbf{h}_{t+1} \leftarrow \mathbf{h}_t \oplus \mathcal{T}_{\text{edit}}(a_t \mid \mathbf{m}_t)$
- 15: Append the addressed evidence and status to obtain $\mathcal{G}_{t+1}$
- 16: Update run history $\mathcal{B}_{t+1} \leftarrow \mathcal{B}_t \cup \{(\mathbf{h}_t, \tilde{\mathbf{c}}_t, \mathbf{c}_t, \mathbf{m}_t, a_t, \eta_t)\}$
- 17: **if** $\eta_t = 1$ and $\mathbf{c}_t$ improves the configured validation score **then**
- 18: $(\mathbf{h}_{\text{best}}, \mathbf{c}_{\text{best}}) \leftarrow (\mathbf{h}_t, \mathbf{c}_t)$
- 19: **end if**
- 20: **end for**
- 21: **return** $(\mathbf{h}_{\text{best}}, \mathbf{c}_{\text{best}})$ and the recorded trajectory $(\mathcal{G}_T, \mathcal{B}_T)$

Table 5: Notation used in CellScientist.

Symbol	Description
$\mathcal{H}$	Model-design space containing task-specific representation, fusion, objective, and training choices
$\mathcal{I}$	Space of executable candidate programs
$\mathcal{D}_{\text{spec}}$	Fixed task specification $(\mathcal{X}, \mathcal{P}, \mathcal{Y}, \mathcal{S}, \mathcal{E})$
$\mathcal{X}, \mathcal{P}, \mathcal{Y}$	Cellular inputs, perturbation descriptors, and response targets
$\mathcal{S}$	Data-partition protocol used by the configured task
$\mathcal{E}$	Evaluation functional used for candidate scoring and selection
$\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}}$	Training, validation, and (when specified) separate reporting views induced by $\mathcal{S}$
$\mathbf{h}_t$	Model-design record at iteration t
$\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$	Structured task-state graph used for routing context and audit
$\mathcal{V}_t, \mathcal{E}_t$	Task nodes and directed dependency edges
$\mathcal{B}_t$	Run history storing attempted designs, implementations, and outcomes
$\mathcal{Z}_t = (\mathbf{h}_t, \mathcal{G}_t, \mathcal{B}_t)$	Complete controller state at iteration t
T, R	Candidate-iteration budget and bounded LCA repair budget
Φ	LLM-conditioned candidate-proposal operator
$\Gamma_{\mathcal{D}}$	Code-application, configured-check, bounded-repair, and rejection procedure
$\tilde{\mathbf{c}}_t$	Proposed candidate edits before execution checks
$\mathbf{c}_t$	Candidate produced after code application and checks
$\perp$	Candidate that failed configured execution or required-output checks
η_t	Indicator that $\mathbf{c}_t$ passed the configured eligibility checks
Ω_{const}	Candidates that pass the implemented checks under $\mathcal{D}_{\text{spec}}$
$\chi_1, \mathcal{C}_1$	Eligibility indicator and its configured implementation-rule set
$\mathcal{F}$	Feedback extraction operator
$\mathbf{m}_t$	Structured feedback vector at iteration t
ψ_k	Individual diagnostic signal used to construct $\mathbf{m}_t$
K_t	Number of diagnostic signals used at iteration t
$\mathcal{A}, a_t$	Diagnostic-to-task association procedure and returned task address
$\mathcal{T}_{\text{edit}}$	Revision-request operator conditioned on recorded feedback
$\oplus$	Application of a revision to a model-design record
$\text{Select}_{\text{val}}$	Selection rule over eligible validation runs
$(\mathbf{h}_{\text{best}}, \mathbf{c}_{\text{best}})$	Validation-selected design record and executable candidate

B DETAILED CASE STUDIES OF AUDITABLE MODEL-REVISION TRAJECTORIES

We reconstruct two model-revision trajectories on morphology-based perturbation-response tasks. Each record preserves execution errors, rule checks, validation outcomes, rejected candidates, and retained candidates under a fixed evaluation protocol. Phase 1–3 follow the pipeline logs: diagnosis and task setup, initial candidate generation, and subsequent review. HRT, LCA, and PDR operate throughout these phases.

On BBBC036, CellScientist retained a Drug–Dose Multiplicative Interaction Architecture (DD-MIA) candidate after evaluating several representation and fusion variants. The selected design combines graph-based compound encoding, batch-related regularization, and dose-aware multiplicative fusion. The trajectory supports selecting this combination under the task-specific validation protocol.

On BBBC047, modulation and VAE-style candidates showed both gains and regressions. The trajectory recorded possible underuse of the supplied control-reference profile as a working diagnostic. CellScientist subsequently retained Ref-RGAF, which adds a deterministic control-reference encoding to the decoder. The record links this diagnostic to a local revision and its validation outcome.

These cases make CellScientist’s selection process inspectable: the records identify attempted assumptions, failed or regressive branches, and validation-selected executable candidates. They complement the quantitative results by tracing local revisions and selection decisions under a fixed protocol.

B.1 DETAILED CASE STUDY A

Case A: Auditable Revision Toward DD-MIA

Trajectory including negative results, crashes, regressions, and retained candidates.

Case Study Overview

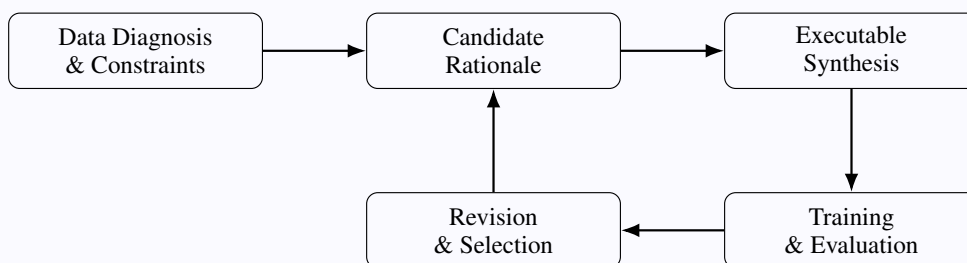
This section reconstructs the model-revision trajectory of **CellScientist** on BBBC036 under the SMILES-based split. The trajectory moves from simple feature-based modeling toward **DD-MIA**, a candidate architecture that separates compound-structure representation from dose-dependent response scaling.

What this case study illustrates. The trace records how CellScientist diagnosed data and batch-related issues, proposed design rationales, implemented candidates, and retained the highest-scoring eligible candidate under the validation protocol.

Trajectory evidence. The trajectory documents how recorded diagnostics and validation outcomes were associated with local design revisions and empirical candidate selection.

Closed-Loop Revision Trajectory

Auditable closed-loop design record:
diagnosis → executable candidate → validation feedback



Interpretation. The loop records a practical model-design process: *diagnose → propose → implement → test → revise*.

(A) From Representation Choice to Design Rationale The trajectory is non-monotonic: several candidates underperformed despite valid execution, and other candidates failed during execution. The complete record shows which components were retained, revised, or rejected at each step.

Table 9 summarizes representative scored candidates from the trajectory. Graph-based compound representation was retained, while several more complex fusion variants regressed or failed.

Key Observation

The trajectory preserves both validation-retained candidates and negative evidence. Several higher-capacity variants regressed, while the validation rule retained a more structured candidate. This record makes the empirical selection process auditable.

(B) Auditable Model-Revision Trajectory We report the diagnosis and revision phases, including failures and regressions. These records show how the validation rule selected the retained candidate.

Phase 1: Problem Formulation and Data Diagnosis. Before architecture revision, the system defines the prediction target and records operational constraints. Table 6 summarizes the data checks that informed later candidates.

Table 6: Phase 1 Execution Trace: Diagnosis and Constraint Identification.

Step	Activity	Artifact	Design / Operational Insight
1	Data Ingestion & Pairing	HDF5 Database	Operational repair: Resolved HDF5 sizing mismatch by enforcing type and shape alignment. Established the $f(\text{ControlRef}, \text{Chem}, \text{Dose}) \rightarrow \text{TreatedResponse}$ mapping used by later candidates.
2	Statistical Diagnosis	ANOVA/PCA	Operational repair: Corrected a syntax error in the statistical constraint formula. Data cue: Observed a measurable plate-related signal in the feature space, motivating later checks for batch-related shortcuts.
3	Dose-Response Profiling	Volcano Plot	Response cue: Identified features with strong perturbation response and observed non-linear, heterogeneous dose behavior. This suggested that a purely linear response model may be insufficient.
4	Technical-Factor Inspection	Assay-Variable Schematic	Constraint definition: The persistence of plate effects after normalization motivated the use of batch-related regularization in later candidates.

Key Observation

From diagnosis to design. ANOVA/PCA diagnostics associated part of the feature variation with technical factors. This recorded observation motivated batch-related regularization in later candidates.

Phase 2: Generate and Execute. Phase 2 evaluated candidate representations and recorded setup errors and unstable training (Table 7). The highest-scoring eligible candidate used graph-based molecular encoding and exceeded the initial flat-vector candidate in validation PCC. Subsequent designs retained this representation based on the observed comparison.

Table 7: Phase 2 Execution Trace: Structural Representation Trial.

Iter	Strategy	Outcome	PCC	Design Analysis & Operational Event
1	Initial Debug / Baseline	Baseline	0.3172	Operational: Pipeline stabilized through a setup-node repair. Baseline: Established lower-bound performance using standard flat-vector features.
2	EquiMorph-GAT	Retained	0.3660	Design cue: Explicit molecular topology encoding improved over the initial flat-vector candidate in this trace, suggesting that compound-structure representation is useful for this task.
3	MD_AdvN (Adversarial)	Regression	0.3378	Negative result: Adversarial normalization alone obtained a lower validation PCC and was rejected by the validation rule.
4	Innovation Variant	Regression	0.3656	Validation check: A graph-based variant produced similar performance across a second implementation attempt.
5	Baseline Re-test	Regression	0.2827	Regression: The simplified baseline obtained a lower validation PCC in this trace; subsequent candidates retained the structured representation.

Key Observation

Representation cue. The graph-based candidate improved over simplified flat-vector baselines within this recorded trajectory, providing task-specific validation evidence for retaining compound-structure encoding.

Phase 3: Review and Revise. Phase 3 evaluated how to combine the control-reference profile, compound representation, and dose information. Table 8 reports all iterations, including crashes and candidates rejected for lower validation performance.

Table 8: **Phase 3 Full Iteration Trace (Iter 1–10).**

Iter	Strategy	Outcome	PCC	Design Analysis
1	Chem-Modulated FiLM	Regression	0.3626	Candidate assumption: affine modulation is sufficient. Result: validation PCC was lower than the retained graph-based model.
2	Res-EquiMorph + Token	Crash	N/A	Candidate assumption: tokenization may improve interaction modeling. Result: instability caused a crash and prevented a valid score.
3	Tokenized Adversarial	Regression	0.3341	Candidate assumption: finer granularity would improve robustness. Result: the tokenized variant obtained lower validation PCC.
4	DGAN (Gated-Adv)	Improved	0.3666	Design pivot: adversarial regularization was restricted to the morphology branch, while chemistry was used to gate the perturbation response.
5	Res-DGAN	Regression	0.3627	Negative add-on: an additional residual skip produced a lower validation PCC in this run.
6	Attn-DGAN (Attention)	Improved	0.3674	Refinement: cross-attention provided a more structured interaction between morphology and compound features.
7	Dose-Gated Attention	Regression	0.3613	Negative result: directly gating attention by dose reduced validation PCC in this candidate.
8	Dose-Conditioned Res	Crash	N/A	Failure mode: custom dose-conditioned gating introduced shape mismatches and training instability.
9	Projected-Token Attn	Improved	0.3658	Partial recovery: identity initialization helped, while the added parameters produced a comparable validation result.
10	DD-MIA	Retained	0.3688	Selected design: separates compound-structure representation from dose-dependent intensity scaling using multiplicative fusion.

Key Observation

Why failures matter. The record preserves crashes and regressions from direct dose injection into attention and more complex tokenized variants. These outcomes expose the sequence of attempted revisions and validation decisions.

(C) Validation-Selected Design: DD-MIA The retained **DD-MIA** candidate combines three implementation choices; the trajectory evaluates their combination without isolating each component’s contribution.

(1) Structural compound representation. The graph-based representation was retained because it improved validation behavior relative to simpler flat-vector candidates in this trace.

(2) Batch-related regularization. Diagnostics suggested technical variation, motivating an adversarial regularization component intended to reduce reliance on plate-related shortcuts in the selected design.

Failure Mode / Design Risk

Plate or batch signals can become shortcuts under some split settings. The adversarial component was included to address this risk during candidate training.

(3) Chemistry–dose multiplicative factorization. The validation-selected candidate uses the following multiplicative fusion form:

$$\Delta\hat{P} = \text{Chem}(\mathcal{G}) \odot \text{Dose}(d). \quad (5)$$

In this implementation, one learned component is conditioned on compound structure and the other on dose. Validation selected this architectural parameterization within the reported trajectory.

Design Rationale Encoded as Architecture

The factorization encodes a task-specific architectural preference in which compound and dose information interact multiplicatively. Its retention follows the validation evidence recorded in this trajectory.

(D) Evaluation and Trajectory Audit The retained DD-MIA candidate achieved a validation PCC of 0.3688 in this trajectory. The record documents its selection path, including failed candidates, regressions, and retained components under the fixed validation protocol.

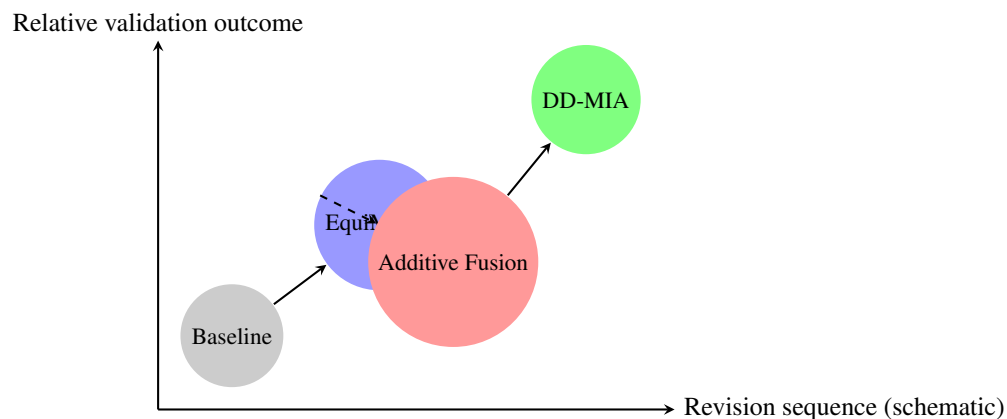


Figure 6: Auditable model-revision trajectory: CellScientist follows a non-monotonic path through candidate architectures, retaining validated components while recording underperforming branches.

Table 9: Trajectory of Candidate Performance.

Activity Step	Design Rationale	PCC	Δ vs Base	Status
Baseline	MLP (Flat Vectors)	0.3396	0.00%	Ref
Initial Search	EquiMorph-GAT	0.3660	+7.77%	Retained candidate
Iter 1	Additive FiLM	0.3626	+6.77%	Regression
Iter 6	Attn-DGAN	0.3674	+8.18%	Improved
Iter 10	DD-MIA	0.3688	+8.60%	Retained candidate

Key Observation

The trajectory records both validation-retained improvements and negative evidence. DD-MIA is the candidate retained for this benchmark under the reported validation-selection rule.

(E) Summary The BBBC036 trajectory connects representation choices, batch-related diagnostics, and feature revisions to recorded execution and validation outcomes. CellScientist retained DD-MIA under the fixed selection rule while preserving failed and regressive alternatives.

B.2 DETAILED CASE STUDY B

The introduction summarizes the BBBC047 trajectory in Figure 2. The detailed record below links its working diagnostics, local revisions, validation outcomes, and retained candidate.

Case B: Auditable Revision Toward Ref-RGAF

Trajectory including negative results, crashes, regressions, and retained candidates.

Case Study Overview

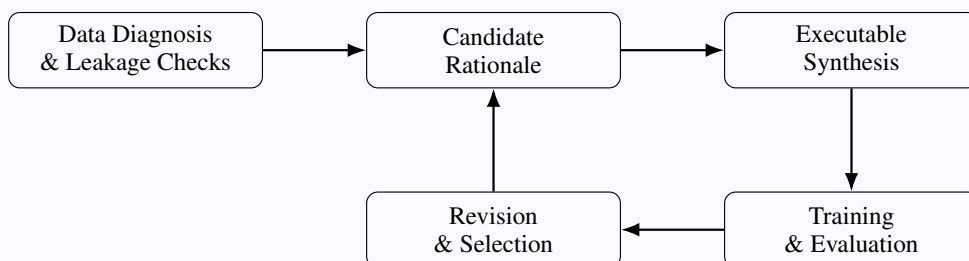
This section reconstructs the model-revision trajectory of **CellScientist** on BBBC047 under the SMILES-based split. The trajectory moves from standard generative baselines toward **Ref-RGAF** (Reference-Gated Affine Fusion), a candidate architecture that adds a deterministic control-reference path to the decoder.

What this case study illustrates. The trace records how the system evaluated modulation-based candidates, observed validation regressions in several VAE-style variants, and retained a reference-aware revision under the fixed validation protocol.

Trajectory evidence. The record links observed validation outcomes to subsequent local revisions and the candidate retained by the selection rule.

Closed-Loop Revision Trajectory

**Auditable closed-loop design record:
candidate revision under validation feedback**



(A) From Regression to Reference-Aware Generation Generative candidates in the BBBC047 trajectory showed both validation gains and regressions. The run recorded possible underuse of the supplied control-reference information as a working diagnostic for subsequent candidate generation.

Key Observation

Observed pivot. The retained revision injects a deterministic encoding h of the supplied control-reference profile alongside the stochastic latent variable in the decoder. The observed validation result supports retaining this reference-aware design.

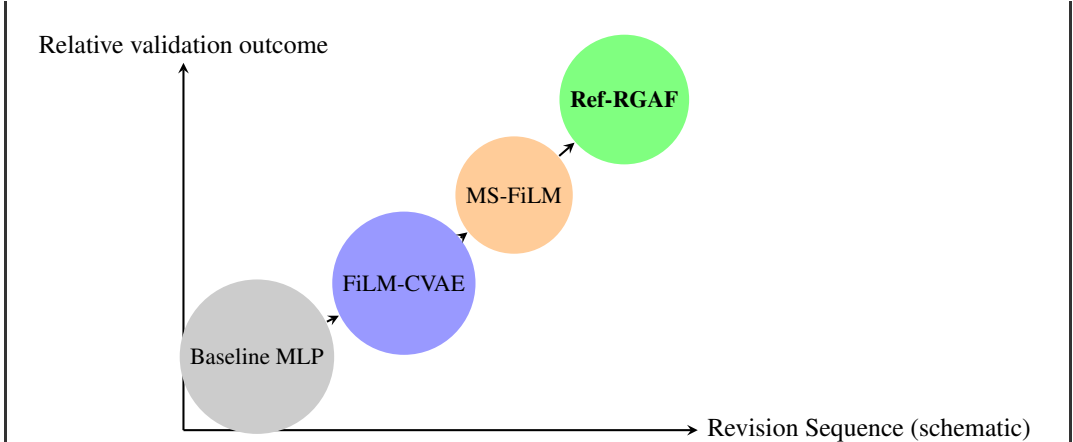


Figure 7: Auditable model-revision trajectory: the system moves from global modulation to multi-scale modulation and then to a reference-aware decoder.

(B) Auditable Model-Revision Trajectory We report the complete trajectory, including Phase 2 instability and Phase 3 revisions, to make validation-based candidate retention inspectable.
Phase 1: Diagnosis and Constraint Identification. The system identified data-quality issues and recorded preprocessing checks and split alignment before candidate evaluation (Table 10).

Table 10: Phase 1 Execution Trace: Diagnosis and Constraint Identification.

Step	Activity	Artifact	Design / Operational Insight
1	Data Ingestion	HDF5/Pandas	Operational repair: Fixed a <code>TypeError</code> caused by mixed numeric and string columns in the raw CSV.
2	Numerical Diagnosis	Feature Scan	Constraint definition: Identified invalid-value warnings and added strict NaN checks and feature filtering to prevent training failures.
3	Split Alignment	Grouped CV	Protocol choice: Grouped samples by exact SMILES identity, matching the SMILES-based split used for the reported controller and trajectory scores.

Phase 2: Generate and Execute. Phase 2 recorded frequent code-generation and training failures. The baseline MLP had the highest validation PCC among eligible candidates and provided the reference for subsequent revisions (Table 11).

Table 11: Phase 2 Execution Trace: Establishing the Baseline.

Iter	Strategy	Outcome	PCC	Design Analysis
1	Init	Crash	N/A	Syntax error in setup node.
2	Baseline MLP	Retained	0.4118	Anchor candidate: A standard MLP using median-normalized features provided a stable reference point.
3	Innovation Var.	Regression	0.3834	Negative result: A more complex candidate obtained a lower validation PCC and was rejected by the validation rule.
4	Training Loop	Repaired / Regression	0.3707	Instability: Training-node failure was repaired, but the candidate degraded.
5	Final Attempt	Repaired / Regression	0.3468	Regression: Severe instability led the system to preserve Iteration 2 as the safe anchor.

Phase 3: Review and Revise. Phase 3 evaluated modulation, gating, and reference-aware variants (Table 12). The retained Ref-RGAF candidate adds a deterministic control-reference path to the generative architecture.

Table 12: **Phase 3 Full Iteration Trace: Revision Logic.** “Improved” denotes an increase from the immediately preceding candidate; “Retained” denotes the best validation-selected candidate in this trace.

Iter	Strategy	Outcome	PCC	Design Analysis
1	FiLM-CVAE	Improved	0.4148	Pivot: Replaced a rigid ODE-style candidate with FiLM modulation, where drug embedding modulates the latent state.
2	Res-FiLM	Regression	0.3416	Negative result: The residual VAE obtained a lower validation PCC; the trace used possible underuse of control-reference information as a working diagnostic for later revisions.
3	Dose-Gated	Improved	0.4095	Candidate assumption: simple dose gating may be sufficient. Result: the candidate obtained a lower validation PCC than the retained anchor.
4	TransMod	Improved	0.4097	The transformer-style fusion candidate obtained a lower validation PCC than the retained anchor.
5	MS-FiLM	Improved	0.4237	Design cue: Applying modulation at multiple decoder layers improved over bottleneck-only modulation.
6	Gated-Attn	Improved	0.4265	Multiplicative gating improved validation PCC relative to the preceding candidates in this trace.
7	RGAF-FiLM	Improved	0.4271	Added affine fusion and a residual hypernetwork.
8	Ref-RGAF	Retained	0.4368	Key revision: Concatenated deterministic control-reference encoding h with latent z ; this candidate obtained the highest validation PCC in the trace.
9	State-Adaptive	Regression	0.4249	The state-adaptive fusion candidate obtained a lower validation PCC than Ref-RGAF.
10	LoRA-FiLM	Regression	0.4163	The LoRA-style latent-rotation candidate obtained a lower validation PCC than Ref-RGAF.

Failure Mode / Design Risk

Working diagnostic. The residual VAE candidate used $y = x + \text{Dec}(z)$ and obtained a lower validation PCC. The run record proposed possible underuse of the supplied control-reference information as a working diagnostic and used it to motivate reference injection.

(C) Validation-Selected Design: Reference-Gated Affine Fusion The retained **Ref-RGAF** candidate combines three implementation choices, described below as design rationales rather than independently established mechanisms.

(1) Control-reference conditioning. Ref-RGAF adds a deterministic reference path:

$$\text{Input to Decoder} = [z_{\text{stochastic}}, h_{\text{deterministic}}]. \quad (6)$$

This allows the decoder to condition its prediction on the supplied plate-matched DMSO-derived control-reference profile.

(2) Affine dose modulation. The drug effect is represented as a dose-conditioned affine transformation:

$$\text{DrugFeat} = (E_{\text{chem}} \odot \sigma(\text{Scale}(d))) + \text{Shift}(d). \quad (7)$$

This gives the model separate scale and shift pathways for response modulation.

(3) Multi-scale modulation. Drug modulation is applied at multiple decoder layers, allowing perturbation effects to enter at several representation levels.

Design Rationale Encoded as Architecture

Design rationale. The retained architecture reflects a task-specific modeling preference for preserving deterministic control-reference information while combining it with compound and dose features.

(D) Validation Performance and Response-Sensitive Metrics The retained candidate achieved five-fold validation PCC 0.4368, using the same evaluation summarized in Table 1. These scores participated in model selection and subsequent revision feedback. Appendix C.1 gives fold values, aggregation, and data roles.

Table 13: Trajectory of Candidate Performance (Phase 3).

Iter	Architecture Strategy	PCC	Δ vs Base	RF-RMSE $\downarrow$	Status
Ref	Baseline MLP	0.4120	0.00%	5.13	Reference
1	FiLM-CVAE	0.4148	+0.68%	5.07	Early gain
5	MS-FiLM	0.4237	+2.84%	5.01	Improved
8	Ref-RGAF	0.4368	+6.02%	4.96	Retained

Key Observation

Interpreting RF-RMSE. Response-feature RMSE measures prediction error on the most perturbation-responsive morphology features, complementing global PCC when evaluating the retained candidate.

(E) Summary On BBBC047, CellScientist used a working diagnostic about control-reference use to propose the Ref-RGAF design. The trajectory preserves valid but underperforming candidates and links the reference-aware revision to execution results and validation-based selection.

C EXPERIMENTAL DETAILS

This appendix details the benchmark tasks, datasets, baselines, metrics, infrastructure, and configuration settings summarized in Section 4.1. It also specifies the evaluation protocols used to obtain the main-text results.

C.1 BENCHMARK TASKS AND DATASETS

CELL PAINTING MORPHOLOGY PREDICTION

We evaluate the CellScientist workflow on three public Cell Painting datasets: BBBC036, BBBC047, and CPG0016. Prediction targets are CellProfiler features (Stirling et al., 2021) that quantify cellular and subcellular properties, including size, shape, texture, and fluorescence intensity. Feature-space readouts provide a common representation for perturbation-response modeling and reduce dependence on pixel-level image-processing choices. Each task includes an assay-context reference derived from vehicle controls according to its preprocessing protocol. This *control-reference profile* represents assay context; it is not a longitudinal pre-treatment measurement of the treated well.

BBBC036 and BBBC047 derive from the Cell Painting small-molecule collection of Bray et al. (Bray et al., 2017). CPG0016 is part of the JUMP Cell Painting resource (Chandrasekaran et al., 2023; Arevalo et al., 2024). All three use U2OS cells and span progressively larger sets of chemical perturbations. Table 14 reports source-profile counts for the processed inputs, distinguishing raw profiles from treated observations used for response prediction. Raw profiles include vehicle controls; treated profiles contain only retained non-DMSO observations. Exact non-control SMILES are counted within each benchmark, and raw feature counts precede split-specific filtering.

The morphology-response benchmarks use two group-based holdout settings, each assigning complete groups to a common fold. The SMILES-based setting groups records by exact recorded SMILES, whereas the plate-based setting groups records by `Metadata_Plate`. These settings evaluate exact-structure and experimental-plate holdout, respectively; neither is a scaffold split.

Table 14: **Source-level statistics of the processed Cell Painting inputs used in our experiments.** Raw profiles include vehicle controls. Treated profiles are non-DMSO observations retained for response prediction. Exact non-control SMILES are counted within each benchmark, and raw Cell Painting feature counts precede split-specific feature filtering.

Dataset	Cell line	Raw profiles	DMSO controls	Treated profiles	Exact non-control SMILES	Plates (all / treated)	Raw CP features
BBBC036	U2OS	18,540	3,528	15,012	1,916	55 / 47	602
BBBC047	U2OS	59,275	16,302	42,973	11,415	241 / 229	894
CPG0016	U2OS	235,307	32,837	202,470	82,282	407 / 392	737

BBBC047 SELECTED-MODEL SCORE PROVENANCE

The BBBC047 SMILES CS-model entry in Table 1 and the Ref-RGAF case study (Figure 2; Table 13) use the same five-fold evaluation. The fold PCC values are 0.451136, 0.432085, 0.431294, 0.433379, and 0.436045. Unrounded records give a mean of 0.4367878232 and a sample standard deviation of 0.0082208836, reported as 0.4368 ± 0.01 in the main table.

For each fold, the other four folds train the candidate and the current fold provides validation for early stopping and checkpoint selection. Mean validation PCC across the five folds compares candidates and feeds subsequent revisions. The search record retains Ref-RGAF at iteration 8 and references its score when proposing iteration 9. Thus, the table summarizes the selected model’s search-used validation performance, and the case study explains how that model was obtained. This provenance record applies to the BBBC047 SMILES CS-model entry. In the controlled study, folds 1–3 fit candidates, fold 4 provides feedback and selection views, and fold 5 supplies separate reporting; that protocol evaluates routing policies independently of the open-workflow score.

LINCS2020 TRANSCRIPTOMIC PERTURBATION PREDICTION

For transcriptomic perturbation prediction, we use gene-expression profiles from the next-generation Connectivity Map (CMap) project, which measures large-scale transcriptional responses with the L1000 platform (Subramanian et al., 2017). Raw and processed profiles are obtained through the LINCS data portal (Stathias et al., 2020), following recommended access and preprocessing protocols (Xie et al., 2022). We construct two benchmark variants to evaluate different generalization regimes: a controlled **Seven Cell Lines** subset and a large-scale **Full Data** benchmark.

The **Seven Cell Lines** subset contains 355 compounds profiled across seven frequently assayed cell lines: A375, HELA, PC3, MCF7, HT29, YAPC, and HA1E. This dense subspace provides high compound–cell coverage and enables a chemical-blind split, where scored compounds are unseen in the corresponding training folds. It therefore measures generalization to held-out compound identities. The **Full Data** benchmark contains 8,316 compounds across 164 cell lines, yielding 78,569 compound–cell expression profiles. This setting is substantially sparser and more heterogeneous and measures performance under long-tail cell-line coverage and diverse perturbation contexts. Dataset statistics are summarized in Table 15.

Table 15: **Statistics of LINCS L1000 benchmarks.**

Dataset	# Compounds	# Cell Lines	# Samples	Characteristics
Seven Cell Lines	355	7	~2,485	Dense; chemical-blind
Full Data	8,316	164	78,569	Sparse; long-tail

C.2 BASELINES

MORPHOLOGY BASELINES

For morphology prediction, we organize baselines into three comparison groups used consistently in the result tables.

Existing reference methods include strong feature-space predictors:

- **TabR** (Gorishniy et al., 2024): A tabular prediction model with a retrieval component that uses nearest-neighbor information at inference time.
- **RealMLP** (Holzmüller et al., 2024): A carefully tuned MLP baseline designed to provide strong and stable performance on tabular benchmarks.
- **RF-TD** (Holzmüller et al., 2024): A Random Forest baseline with tuned default settings for competitive tabular prediction.

Search-based baselines provide generic model-selection reference points under the corresponding benchmark split, target, and metric definitions:

- **FLAML** (Wang et al., 2021): An AutoML framework that searches over model and hyperparameter configurations under a configured budget.
- **Random Search** (Gijssbers et al., 2024): A generic search baseline that samples candidate configurations independently.

Agent-designed baselines test whether an existing automated modeling workflow is sufficient:

- **CellForge** (Tang et al., 2025): A multi-agent automated modeling framework for proposing, training, and selecting predictive models.

These groups compare fixed supervised predictors, generic model-selection procedures, and the CellForge automated perturbation-modeling workflow.

TRANSCRIPTOMIC AND TEMPORAL BASELINES

For LINCS2020 transcriptomic perturbation prediction, we compare against representative domain-specific baselines:

- **DLEPS** (Zhu et al., 2021): A CNN-based model that maps chemical structures to gene-expression changes with cell-line embeddings.
- **DeepCE** (Pham et al., 2021): A graph neural network framework that combines chemical substructure representations with cell-line features.
- **CIGER** (Pham et al., 2022): A collaborative filtering approach that models drug–cell–dose interactions with graph convolutional networks.
- **MultiDCP** (Wu et al., 2022): A multi-view deep learning model with separate drug and cell-line encoders and contrastive fusion.
- **TranSiGen** (Tong et al., 2024): A VAE-based generative model that learns latent representations for denoising and reconstructing perturbation profiles.

C.3 EVALUATION METRICS

MORPHOLOGY METRICS

For morphology prediction, we evaluate both global profile fidelity and perturbation-sensitive feature recovery. Given the stacked prediction and target matrices $\hat{\mathbf{Y}}$ and $\mathbf{Y}$ for an evaluation partition, global PCC is $\text{corr}(\text{vec}(\hat{\mathbf{Y}}), \text{vec}(\mathbf{Y}))$; MSE and R^2 are computed over the corresponding entries. This same global-PCC functional is used in the high-freedom morphology benchmark and the controlled audit, while the fitted predictor, partition aggregation, and experimental replication follow their respective protocols. MSE captures numerical reconstruction error, PCC measures global profile association, and R^2 evaluates explained feature variance; we report all three because high-dimensional morphology profiles can differ in scale, calibration, and variance across datasets.

To further assess whether a model captures the most responsive morphology changes, we compute response-feature metrics on the top- K perturbed features. For each sample i , let $\mathbf{y}_{i,\text{gt}}$ be the ground-truth response vector and $\boldsymbol{\mu}_{\text{ctrl}}$ the mean control profile. We select responsive features using only ground-truth deviation from control:

$$\mathcal{F}_i^{(K)} = \text{topk}_j \left(\left| \mathbf{y}_{i,\text{gt}}^{(j)} - \boldsymbol{\mu}_{\text{ctrl}}^{(j)} \right| \right), \quad K \in \{20, 50\}.$$

We then compute $\text{RMSE}_{\text{RF-}K}$ and $\text{PCC}_{\text{RF-}K}$ within $\mathcal{F}_i^{(K)}$, where RF denotes response feature. The feature subset is determined solely from ground truth and control profiles, independently of model outputs. These metrics provide complementary evaluation views, while PCC remains the configured model-selection metric.

TRANSCRIPTOMIC AND TEMPORAL METRICS

For LINCS2020 transcriptomic prediction, RMSE and PCC measure global expression-profile fidelity. Positive and Negative P@100 measure recovery of the most strongly upregulated and down-regulated genes, respectively. These metrics complement whole-profile accuracy with the recovery of strongly responding genes.

For Schiebinger temporal forecasting, global PCC and MSE quantify agreement between predicted and observed expression over the common held-out evaluation units. The gene order and evaluator are shared across the three predictors (Appendix F.3).

C.4 IMPLEMENTATION DETAILS

IMPLEMENTATION AND EXECUTION CONFIGURATION

Table 16 reports the environments used for the high-freedom workflow and the controlled audit. Methods compared within each experiment use the same hardware and software environment. Table 17 separates open-workflow settings from those used to compare controllers within the fixed candidate language.

Table 16: **Computational infrastructure and software environment.**

Component	Specification
High-freedom workflow GPU	NVIDIA RTX 5880 Ada Generation (48GB VRAM)
High-freedom workflow software	Python 3.11.14; PyTorch 2.0.1+cu118
Controlled-audit GPU	NVIDIA H100 (80GB HBM3)
Controlled-audit software	Python 3.10.20; PyTorch 2.3.1+cu121
Operating system	Linux

CONTROLLED ROUTING POLICY

Candidate language and parent state. The controlled audit uses configurations $c = (m, t, r_x, r_y, \alpha)$, where $m \in \{\text{pre}, \text{pre_dose}, \text{pre_smiles}, \text{pre_smiles_dose}\}$, $t \in \{\text{direct}, \text{delta}\}$, $r_x, r_y \in \{16, 32, 64, 128\}$, and $\alpha \in \{0.01, 0.1, 1, 10, 100, 1000\}$. Their Cartesian product contains 768 candidates. The five fields correspond, respectively, to conditioning, target, representation, decoder, and regularization addresses. A legal atomic edit changes exactly one field. At each revision, the parent c^* is the previously evaluated candidate with highest feedback global PCC, with the earliest observation retained on an exact tie. Let g , r , and q denote its feedback global PCC, response PCC, and top-50 response PCC. For capacity address a , define G_a to hold when history contains a candidate with lower capacity at a , identical values at all other fields, and lower feedback global PCC than c^* .

Ordered rules and history filtering. Starting from an empty list, append the following addresses in order: (i) conditioning if $m \neq \text{pre_smiles_dose}$; (ii) target if $t \neq \text{delta}$ or $r > g + 0.05$; (iii) decoder if $q < r$; (iv) representation if $r_x < 64$ or $(r_x = 64 \wedge G_{\text{representation}})$; (v) decoder if $r_y < 64$ or $(r_y = 64 \wedge G_{\text{decoder}})$; and (vi) regularization, target, conditioning, in that order. Repeated addresses retain their first position. Remove an address if a trial at that address after c^* was evaluated failed to exceed its feedback score, or if it has no unseen one-field neighbor. This history filter uses the current parent’s evaluation step as its cutoff; an improved incumbent can make an address eligible again. The decoder response rule in (iii) is distinct from the capacity evidence rule in (v). Retain the first two remaining addresses, or the sole remaining address when only one is available.

Table 17: CellScientist controller and execution settings.

Configuration	Value	Role in the workflow
Open-workflow backbone	Gemini 3 Pro	Default backbone for high-freedom model exploration
Open-workflow context length	40,000 tokens	Maintained interaction context limit
Open-workflow temperatures	0.5 analysis; 0.7 proposal	Sampling settings for analysis and generative revision
Open-workflow prompt views	4	Robust scaling, PCA, feature-network, and enrichment views
Open-workflow iteration budget	10 revisions	Maximum number of high-freedom revision steps
Open-workflow repair budget	3 rounds	Configured debugging budget for runtime or interface errors
Controlled-audit backbones	Gemini 3 Pro; Qwen2.5-0.5B-Instruct	Manuscript-default and open-weight reproduction
Controlled-audit decoding	Temperature 0	Fixed within every matched controller comparison
Controlled-audit candidate language	768 executable candidates	Shared predictor configurations available to every policy
Controlled-audit fitting	Seeded randomized low-rank fitting	Identical fitting seed for every policy within each paired run
Controlled-audit budgets	$B \in \{1, 3, 5, 10\}$	Trained and evaluated candidates, including h_0
Validation selection metric	Global PCC	Metric used by $\text{Select}_{\text{val}}$ in the controlled audit
Controlled-audit retention view	Mean feedback/selection PCC	Identical candidate-retention rule for every matched policy

Deterministic edits and LLM choice. Each retained address receives one unseen legal neighbor using a fixed preference order: conditioning uses `pre_smiles_dose`, `pre_smiles`, `pre_dose`, `pre`; target uses `delta`, `direct`; both rank fields use 64, 128, 32, 16; and regularization uses 1, 0.1, 0.01, 10, 100, 1000. The current field value is excluded, as are neighbors whose full candidate ID has already been evaluated. Candidate ID breaks any remaining tie. The LLM sees the current configuration, feedback metrics, ordered addresses, and these instantiated address–candidate pairs. Its instruction prefers the first-ranked pair unless the feedback supports the second. It returns one listed pair; it neither generates a new configuration nor searches the full candidate space. Parsing and candidate/address consistency are checked deterministically. A failed request or invalid decision uses the first listed pair and is recorded. If no ranked address remains, a separate logged runner-level recovery chooses an unseen one-field neighbor deterministically from the run seed and step; if none remains, search terminates. Full history informs the preceding parent-selection, filtering, and capacity rules rather than being copied in full into the LLM prompt.

Comparison and implementation. Random routing draws uniformly from all addresses with unseen legal neighbors, then applies the same deterministic address-specific edit instantiation. It does not use CellScientist’s diagnostic shortlist or history-based suppression. The comparison therefore tests the combined routing policy under a shared candidate language and evaluator. The implementation is specified by `CellScientistController.rank_addresses`, `CellScientistController.propose`, and `_canonical_atomic_edit` in `cellscientist/controllers.py`, together with `CandidateSpace.neighbors` in `cellscientist/search_space.py`.

Feedback, selection, and reporting. The primary controlled configuration starts from `(pre_dose, delta, 32, 32, 10)`. Folds 1–3 fit candidates. Fold-4 groups are ordered deterministically by SHA256 and allocated alternately to disjoint feedback and selection views; row and group separation are checked across all views. Proposal decisions use feedback metrics only. After search, candidates in each budget prefix are scored on the selection view, and the candidate maximizing mean feedback/selection global PCC is retained. Fold 5 then provides the reporting score. This distinguishes the feedback-selected routing parent from the finally retained candidate; neither decision accesses reporting outcomes. Each budget B includes the initial candidate h_0 .

Statistical aggregation. Table 3 summarizes five paired seeds within each task/split setting. Between-policy effects are first averaged over seeds within each of the four settings; paired-setting 95% t -confidence intervals use these four effects. The normalized frontier-AUC improvements over Fixed h_0 , Flat retry, Random routing, and AIDE-style search are 0.0877, 0.0608, 0.0237, and 0.0215, with respective intervals [0.0680, 0.1074], [0.0414, 0.0802], [0.0172, 0.0303], and [0.0117, 0.0314]. This aggregate measures performance across the tested candidate-evaluation budgets; the budget-specific random-routing difference is unresolved at $B = 10$.

PROTOCOL CONTROLS AND ROUTING DETAILS

This section specifies evaluation semantics and feedback routing for the high-freedom workflow. Within each benchmark, methods use the reported split, target, and metric definitions. Fixed predictors follow their model protocols, search baselines use configured search procedures, and CellForge follows its agent workflow. CellScientist uses 10 revision steps with up to 3 local implementation repairs per step. Candidates are eligible for selection only after passing the configured checks and completing scoring; rejected and failed candidates remain in the run history.

PDR associates structured execution and validation diagnostics with proposed edits to task components. Diagnostic records identify one or more edit targets; LCA then checks the resulting edits against the configured task contract. Table 18 summarizes representative mappings. Listing 1 gives a normalized illustration reconstructed from the BBBC047 trajectory.

Table 18: **Representative manuscript-level PDR mappings.** PDR maps diagnostic signals to proposed local edit targets; LCA checks the resulting edits against protected evaluation semantics.

Diagnostic	Editable region	Protected semantics
Interface failure	Data adapter or implementation interface	Split, target, and metric definitions
Metric-reporting failure	Evaluation adapter	Metric formulas and fold protocol
Validation collapse	Fusion, decoder, objective, or training node	Dataset split, target definition, and output space
Weak perturbation sensitivity	Perturbation encoding, dose encoding, or fusion edge	Evaluation code and response-feature definition
Suspected underuse of control reference	Encoder state, decoder input, or fusion component	Preprocessing, split, target, and metrics
Batch-shortcut risk	Representation or training-regularization component	Configured split and evaluation rules

```
{
  "dataset": "BBBC047",
  "iteration": 2,
  "diagnostic": {
    "type": "suspected_control_reference_underuse",
    "candidate": "Res-FiLM",
    "execution_status": "valid",
    "rule_status": "passed",
    "selection_metric": "PCC",
    "candidate_score": 0.3416,
    "best_previous_score": 0.4148,
    "verdict": "valid_but_regressed",
    "summary": "Validation PCC dropped after a residual VAE-style
      update; control-reference use was recorded as a working
      diagnostic."
  },
  "routed_address": {
    "address_type": "proposed_edit_targets",
    "editable_components": [
      "model_architecture",
      "encoder_state_representation",

```

```

        "decoder_input",
        "fusion_module"
    ],
    "routing_reason": "valid execution with degraded validation
        performance and a working diagnostic about control-reference use"
},
"allowed_edit": {
    "operation": "test_reference_aware_decoder_path",
    "protected_components": [
        "dataset_split",
        "target_definition",
        "feature_preprocessing",
        "validation_protocol",
        "evaluation_metrics",
        "fold_protocol"
    ]
},
"revised_candidate": {
    "name": "Ref-RGAF",
    "design_change": "concatenate deterministic control-reference
        encoding h with stochastic latent code z before decoding",
    "intended_effect": "retain supplied control-reference information
        while allowing compound-dose modulation"
},
"lca_check": [
    "interface_check",
    "shape_check",
    "leakage_check",
    "metric_completeness"
]
}

```

Listing 1: Normalized manuscript-level illustration reconstructed from the BBBC047 trajectory.

LLM BACKBONES

The matched-budget audit uses Gemini 3 Pro for the manuscript-default runs and repeats the controller comparison with the open-weight Qwen2.5-0.5B-Instruct model. Within each backbone, every controller receives the same decoding configuration and candidate budget. Table 19 records the public model identities used in this audit.

Controller-call accounting. Gemini completed all 720 logical controller decisions with valid allow-listed responses and no fallback. Three transient transport attempts were retried successfully and remain included in request accounting. Qwen completed all 720 local HTTP requests successfully. Its schema validator invoked the registered legal fallback for 10/180 CellScientist requests and 70/180 AIDE-style requests after non-allow-listed field pairings. Flat and Random routing required no fallback. All such events remain in the reported trajectories and candidate budgets.

Table 19: **LLM backbones used in the controlled framework audit.** Gemini 3 Pro is the manuscript-default backbone; Qwen2.5-0.5B-Instruct provides an open-weight reproduction.

Main-text Label	Full Backbone Identity	Provider / Family	Source Link
Gemini 3 Pro	Gemini 3 Pro	Google Gemini	https://blog.google/products/gemini/gemini-3
Qwen 0.5B	Qwen2.5-0.5B-Instruct	Qwen	https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct

D CONTROLLER-BACKBONE REPRODUCTION

Table 20: **Task-level reproduction of structured routing across controller backbones.** The upper panel reports mean±sample standard deviation in held-out global PCC over the same five paired controller and fitting seeds. The lower panel reports CellScientist-minus-Random effects with paired-setting 95% *t*-confidence intervals across the four task/split settings, computed from unrounded run-level values; confidence intervals are shown in smaller type. Gemini 3 Pro and the open-weight Qwen2.5-0.5B-Instruct use the same settings, candidate language, fitting procedure, and evaluation budgets. Bold marks the better mean within each backbone, setting, and budget.

Backbone	Dataset	Split	B = 3		B = 5		B = 10	
			Random	CellScientist	Random	CellScientist	Random	CellScientist
Gemini 3 Pro	BBBC036	Plate	0.2250 $\pm$ 0.0493	0.2650 $\pm$ 0.0005	0.2277 $\pm$ 0.0502	0.2674 $\pm$ 0.0003	0.2626 $\pm$ 0.0077	0.2693 $\pm$ 0.0012
	BBBC036	SMILES	0.2052 $\pm$ 0.0413	0.2336 $\pm$ 0.0016	0.2049 $\pm$ 0.0417	0.2346 $\pm$ 0.0009	0.2347 $\pm$ 0.0016	0.2346 $\pm$ 0.0009
	BBBC047	Plate	0.3032 $\pm$ 0.0500	0.3396 $\pm$ 0.0004	0.3066 $\pm$ 0.0514	0.3457 $\pm$ 0.0004	0.3458 $\pm$ 0.0027	0.3481 $\pm$ 0.0009
	BBBC047	SMILES	0.3131 $\pm$ 0.0536	0.3529 $\pm$ 0.0010	0.3207 $\pm$ 0.0582	0.3614 $\pm$ 0.0003	0.3668 $\pm$ 0.0073	0.3751 $\pm$ 0.0017
Qwen2.5-0.5B	BBBC036	Plate	0.2250 $\pm$ 0.0493	0.2566 $\pm$ 0.0024	0.2277 $\pm$ 0.0502	0.2566 $\pm$ 0.0024	0.2626 $\pm$ 0.0077	0.2657 $\pm$ 0.0035
	BBBC036	SMILES	0.2052 $\pm$ 0.0413	0.2357 $\pm$ 0.0004	0.2049 $\pm$ 0.0417	0.2357 $\pm$ 0.0004	0.2347 $\pm$ 0.0016	0.2346 $\pm$ 0.0009
	BBBC047	Plate	0.3032 $\pm$ 0.0500	0.3429 $\pm$ 0.0003	0.3066 $\pm$ 0.0514	0.3425 $\pm$ 0.0009	0.3458 $\pm$ 0.0027	0.3438 $\pm$ 0.0018
	BBBC047	SMILES	0.3131 $\pm$ 0.0536	0.3570 $\pm$ 0.0005	0.3207 $\pm$ 0.0582	0.3615 $\pm$ 0.0005	0.3668 $\pm$ 0.0073	0.3707 $\pm$ 0.0002
<i>CellScientist – Random: mean effect with paired-setting 95% CI</i>								
Backbone	Contrast		B = 3		B = 5		B = 10	
Gemini 3 Pro	CellScientist – Random		+0.0361 [+0.0275,+0.0448]		+0.0373 [+0.0292,+0.0454]		+0.0043 [-0.0018,+0.0104]	
Qwen2.5-0.5B-Instruct	CellScientist – Random		+0.0364 [+0.0261,+0.0467]		+0.0341 [+0.0256,+0.0426]		+0.0012 [-0.0032,+0.0056]	

E DETAILED LCA AND PDR AUDIT RESULTS

Table 4 summarizes the HRT, LCA, and PDR results in the main text. Here we report LCA outcomes and PDR results by component address, including registered case counts, decisions, localization, repair success, unrelated edits, and repair rounds.

E.1 REGISTERED LCA OUTCOME AUDIT

The LCA audit includes 40 registered violation cases and 20 legal controls. Each positive case contains one prespecified violation, and each legal control contains none. LCA repaired 15 pre-run cases, blocked 15 pre-run violations, and rejected 10 post-run violations. Legal controls remained unchanged, yielding 60/60 correct decisions and 40/40 contained violations. Thirty invalid evaluations were avoided before execution.

E.2 HELD-OUT REGISTERED-FAULT PDR AUDIT

The PDR audit uses five calibration cases and 15 held-out faults, with the split fixed before evaluation. Each policy is evaluated on the same faults, with three cases per component address. Flat retry and Random routing provide comparison policies for localization and repair. Structured PDR achieved 100% Top-1 and Top-2 localization at every address. It completed every repair with 0 unrelated edits and 1.00 mean repair round.

Table 21: **Registered LCA audit grouped by outcome.** Each positive case contains one prespecified violation and each legal control contains none. The table separates violations that can be restored to an eligible candidate from violations that must be safely rejected before or after execution. “Eligible” denotes eligibility after LCA, and “Avoided” counts invalid evaluations avoided. Across the 40 positive and 20 negative cases, violation recall, exact-code accuracy, legal-case specificity, and repair-or-safe-block containment are all 100%.

LCA outcome	Registered cases	Stage	Cases	Detected / accepted	Observed action	Eligible	Avoided
Repair	Protected-state mutation; interface mismatch; route mismatch	Pre	15	15/15 detected	15/15 repaired	15/15	15
Pre-run block	Out-of-domain candidate; non-atomic edit; repair budget exhausted	Pre	15	15/15 detected	15/15 blocked	0/15	15
Post-run block	Output-shape mismatch; non-finite output	Post	10	10/10 detected	10/10 blocked	0/10	0
Legal controls	Contract-preserving candidates across registered interfaces and component domains	Pre	20	20/20 accepted	Unchanged	20/20	0
Overall	All registered violation and legal-control cases		60	60/60 correct	40/40 contained	35/60 eligible	30

Table 22: **Held-out registered-fault PDR audit by component address.** The fault-to-address split is frozen before evaluation: five cases are used for calibration and the remaining 15 for reporting. Each held-out address has three registered cases. Structured PDR uses Gemini 3 Pro to select an allow-listed address from the diagnostic shortlist. Unrepaired cases are censored at the five-round repair budget. Best values within each address are **bolded**; ties share emphasis.

Routing policy	Expected address	Cases	Top-1 accuracy ↑	Top-2 recall ↑	Repair success ↑	Mean unrelated edits ↓	Mean repair rounds ↓
Flat retry	Conditioning	3	66.7%	N/A	0.0%	4.33	5.00
	Response parameterization	3	0.0%	N/A	100%	1.00	2.00
	Representation	3	0.0%	N/A	0.0%	4.00	5.00
	Decoder	3	0.0%	N/A	0.0%	4.33	5.00
	Regularization	3	0.0%	N/A	66.7%	2.33	3.67
	All held-out	15	13.3%	N/A	33.3%	3.20	4.80
Random routing	Conditioning	3	0.0%	0.0%	0.0%	5.00	5.00
	Response parameterization	3	33.3%	33.3%	66.7%	2.67	3.33
	Representation	3	0.0%	0.0%	66.7%	3.33	4.00
	Decoder	3	0.0%	33.3%	100%	2.33	3.33
	Regularization	3	0.0%	0.0%	66.7%	3.00	3.67
	All held-out	15	6.7%	13.3%	60.0%	3.27	4.27
Structured PDR	Conditioning	3	100%	100%	100%	0.00	1.00
	Response parameterization	3	100%	100%	100%	0.00	1.00
	Representation	3	100%	100%	100%	0.00	1.00
	Decoder	3	100%	100%	100%	0.00	1.00
	Regularization	3	100%	100%	100%	0.00	1.00
	All held-out	15	100%	100%	100%	0.00	1.00

F ADDITIONAL EXPERIMENTAL ANALYSIS

This appendix characterizes the high-freedom workflow, evaluates recovery of responsive morphology features, and reports temporal forecasting on Schiebinger. These analyses complement the main results under their respective evaluation protocols.

F.1 END-TO-END OPEN-WORKFLOW CHARACTERIZATION

Open-workflow evidence. Tables 1 and 2 report task-specific predictors selected by the high-freedom CellScientist workflow under morphology and transcriptomic protocols. Figures 4 and 5 show fold-level comparisons and non-monotonic revision trajectories, respectively. These results assess predictor quality; the main-text RQ2 audit separately compares controller policies within a fixed candidate language.

The RQ2 study compares controller policies within a fixed executable language. Table 23 complements that audit with an end-to-end comparison in which implementations and selected model families can vary. Cumulative configurations share the task specification, split, validation metric, iteration budget, repair budget, execution timeout, and environment. The comparison records execution success, rule-compliant success, predictive quality, and runtime as structured task state, configured checks, and history-conditioned revision are added.

Table 23: **End-to-end open-workflow characterization.** SR denotes execution success, R-SR denotes rule-compliant success, and Avg/Best PCC summarize validation performance over eligible runs. Time reports wall-clock runtime. This high-freedom characterization complements the fixed-language comparisons in RQ2.

ID	Method Configuration	BBBC036 (SMILES-based Split)					BBBC047 (SMILES-based Split)				
		SR $\uparrow$	R-SR $\uparrow$	Avg PCC $\uparrow$	Best PCC $\uparrow$	Time (h) $\downarrow$	SR $\uparrow$	R-SR $\uparrow$	Avg PCC $\uparrow$	Best PCC $\uparrow$	Time (h) $\downarrow$
M0	Prompt + LLM + Code Executor	0.526	0.526	0.1357	0.1682	10.25	0.368	0.368	0.1422	0.1752	15.58
M1	M0 + Re-prompting (Naive)	0.263	0.263	0.2156	0.2485	17.96	0.210	0.210	0.2267	0.2581	27.29
M2	M1 + Structured Task State (HRT)	0.947	0.684	0.2771	0.3050	10.34	0.941	0.588	0.2472	0.2775	15.96
M3	M2 + Protected Editing / Checks (LCA)	1.000	0.737	0.3183	0.3214	8.46	1.000	0.684	0.3315	0.3962	13.38
M4	M3 + History-Conditioned Revision (PDR)	1.000	0.789	0.3413	0.3688	7.93	1.000	0.714	0.3539	0.4368	12.58

Table 24: **Response-sensitive morphology metrics.** Response-feature metrics evaluate prediction quality on the top- K most responsive morphology features. CS-model has the best mean values among the listed methods under both SMILES-based and plate-based splits in this table. Best mean values are **bolded**.

Method	Venue	SMILES-based Split				Plate-based Split			
		RMSE _{RF-20} ↓	RMSE _{RF-50} ↓	PCC _{RF-20} ↑	PCC _{RF-50} ↑	RMSE _{RF-20} ↓	RMSE _{RF-50} ↓	PCC _{RF-20} ↑	PCC _{RF-50} ↑
BBBC036									
# Existing reference methods									
TabR	ICLR'24	4.1306 _{±0.03}	3.3883 _{±0.04}	0.2742 _{±0.01}	0.2584 _{±0.01}	3.8974 _{±0.12}	3.1610 _{±0.07}	0.3784 _{±0.05}	0.3783 _{±0.05}
RealMLP	NeurIPS'24	3.9910 _{±0.03}	3.2980 _{±0.03}	0.3170 _{±0.01}	0.2929 _{±0.01}	3.5303 _{±0.13}	2.8547 _{±0.08}	0.4636 _{±0.03}	0.4681 _{±0.01}
RF-TD	NeurIPS'24	4.1499 _{±0.02}	3.4356 _{±0.03}	0.2445 _{±0.01}	0.2227 _{±0.01}	3.9590 _{±0.13}	3.2927 _{±0.07}	0.5268 _{±0.05}	0.4649 _{±0.05}
□ Search-based baselines									
FLAML	MLSys'21	3.9892 _{±0.04}	3.2897 _{±0.04}	0.3297 _{±0.01}	0.3079 _{±0.01}	3.3658 _{±0.11}	2.7702 _{±0.08}	0.6486 _{±0.03}	0.6705 _{±0.03}
Random Search	JMLR'24	3.9948 _{±0.05}	3.2964 _{±0.05}	0.3218 _{±0.01}	0.2978 _{±0.01}	3.4316 _{±0.12}	2.8357 _{±0.08}	0.6235 _{±0.04}	0.6468 _{±0.04}
△ Agent-designed methods									
CellForge	arXiv'25	3.9903 _{±0.05}	3.2928 _{±0.05}	0.3262 _{±0.01}	0.3034 _{±0.01}	3.3869 _{±0.10}	2.7883 _{±0.08}	0.6574 _{±0.03}	0.6813 _{±0.03}
CS-model	Ours	3.9865 _{±0.09}	3.2849 _{±0.08}	0.3339 _{±0.01}	0.3139 _{±0.01}	3.2817 _{±0.11}	2.7046 _{±0.09}	0.6856 _{±0.02}	0.7082 _{±0.02}
BBBC047									
# Existing reference methods									
TabR	ICLR'24	6.2132 _{±0.02}	4.6263 _{±0.01}	0.3355 _{±0.01}	0.3898 _{±0.01}	6.2120 _{±0.05}	4.6035 _{±0.04}	0.3398 _{±0.01}	0.4009 _{±0.01}
RealMLP	NeurIPS'24	5.8479 _{±0.02}	4.4168 _{±0.01}	0.3678 _{±0.01}	0.4851 _{±0.01}	6.1791 _{±0.06}	4.5301 _{±0.04}	0.3836 _{±0.02}	0.4270 _{±0.01}
RF-TD	NeurIPS'24	6.0601 _{±0.03}	4.5652 _{±0.02}	0.3742 _{±0.01}	0.4153 _{±0.01}	6.0934 _{±0.05}	4.5715 _{±0.04}	0.3591 _{±0.02}	0.4063 _{±0.01}
□ Search-based baselines									
FLAML	MLSys'21	5.2847 _{±0.04}	4.0209 _{±0.03}	0.3685 _{±0.01}	0.4979 _{±0.01}	5.5128 _{±0.05}	4.0879 _{±0.04}	0.3892 _{±0.02}	0.5096 _{±0.01}
Random Search	JMLR'24	5.4362 _{±0.05}	4.1328 _{±0.04}	0.3601 _{±0.01}	0.4887 _{±0.01}	5.6315 _{±0.06}	4.1746 _{±0.04}	0.3838 _{±0.02}	0.4983 _{±0.01}
△ Agent-designed methods									
CellForge	arXiv'25	5.1164 _{±0.04}	3.8946 _{±0.03}	0.3748 _{±0.01}	0.5096 _{±0.01}	5.3846 _{±0.05}	3.9907 _{±0.04}	0.3967 _{±0.02}	0.5208 _{±0.01}
CS-model	Ours	4.9589 _{±0.03}	3.7581 _{±0.03}	0.3796 _{±0.01}	0.5172 _{±0.01}	5.2274 _{±0.04}	3.8788 _{±0.04}	0.4025 _{±0.02}	0.5312 _{±0.01}
CPG0016									
# Existing reference methods									
TabR	ICLR'24	2.6335 _{±0.05}	2.3924 _{±0.05}	0.0896 _{±0.01}	0.0865 _{±0.01}	2.5259 _{±0.01}	2.2874 _{±0.01}	0.1330 _{±0.01}	0.1309 _{±0.01}
RealMLP	NeurIPS'24	2.5152 _{±0.04}	2.2456 _{±0.04}	0.3770 _{±0.02}	0.3579 _{±0.02}	2.3456 _{±0.01}	2.0846 _{±0.01}	0.4658 _{±0.01}	0.4504 _{±0.01}
RF-TD	NeurIPS'24	2.5885 _{±0.05}	2.3130 _{±0.04}	0.0170 _{±0.01}	0.0224 _{±0.01}	2.4955 _{±0.01}	2.2221 _{±0.01}	0.0758 _{±0.01}	0.0835 _{±0.01}
□ Search-based baselines									
FLAML	MLSys'21	2.1126 _{±0.03}	1.9075 _{±0.03}	0.5062 _{±0.01}	0.4974 _{±0.01}	2.0249 _{±0.02}	1.8186 _{±0.02}	0.5614 _{±0.02}	0.5537 _{±0.02}
Random Search	JMLR'24	2.2484 _{±0.04}	2.0293 _{±0.04}	0.4385 _{±0.02}	0.4262 _{±0.02}	2.1227 _{±0.02}	1.9084 _{±0.02}	0.5126 _{±0.02}	0.5028 _{±0.02}
△ Agent-designed methods									
CellForge	arXiv'25	2.1038 _{±0.03}	1.8937 _{±0.03}	0.4978 _{±0.01}	0.4896 _{±0.01}	1.9426 _{±0.02}	1.7438 _{±0.02}	0.5867 _{±0.03}	0.5789 _{±0.03}
CS-model	Ours	1.8127 _{±0.02}	1.6328 _{±0.01}	0.5924 _{±0.01}	0.5890 _{±0.01}	1.7303 _{±0.02}	1.5517 _{±0.02}	0.6529 _{±0.03}	0.6518 _{±0.03}

On both datasets, M4 achieved the highest R-SR, Avg PCC, and Best PCC, and the lowest runtime among the tested open-workflow configurations. Figure 5 records exploratory regressions and validation-retained improvements, linking these aggregate results to the case-study trajectories.

F.2 RESPONSE-SENSITIVE MORPHOLOGY METRICS

Table 24 describes the observed method differences on the most perturbation-responsive morphology features. We report response-feature RMSE and PCC over the top- K responsive features on BBBC036, BBBC047, and CPG0016 under both SMILES-based and plate-based splits.

F.3 SCHIEBINGER TEMPORAL FORECASTING

The Schiebinger dataset (Schiebinger et al., 2019) profiles fibroblast-to-iPSC reprogramming across 39 labelled time points from day 0 to day 18. Of its 251,203 expression profiles, 236,285 have matched day metadata and define the temporal protocol. The processed inputs contain 1,479 RNA genes and four branch–replicate trajectories.

We compare the selected CS-model against scNODE (Zhang et al., 2024), which combines a variational autoencoder with a neural ordinary differential equation, and a MIOFlow adaptation (Huguet et al., 2022), which combines a geodesic autoencoder, optimal transport, and a neural ordinary differential equation. All methods share the same trajectories, gene order, deterministic 512-cell evaluation units, and evaluator. Candidates are selected using only days 14.5–15.5 and refit through day 15.5; final evaluation uses 20 untouched day-16–18 units. Table 25 reports global PCC and MSE under this common protocol.

Table 25: **Schiebinger common-protocol temporal forecasting.** The CS-model selected by CellScientist and two temporal baselines share inputs, the development interval, refit cutoff, held-out evaluation units, gene order, and evaluator. Best values among these predictors are **bolded**.

Method	Global PCC $\uparrow$	MSE $\downarrow$
CS-model	0.9655	0.04128
scNODE	0.9523	0.06040
MIOFlow adaptation	0.8935	0.11787